UME

Organization / Business Operating System

Low-Level Software Design Specification

Document ID: UME-LLD-001
Version: 1.0.0
Status: Draft — Baseline
Date: March 2026
Classification: Internal — Confidential
Language: Rust (platform) · Java (client suite)
Coverage: Kernel · 42 Org Modules · API Layer · Data Architecture · Security · Deployment · SDK

Executive Summary

This Low-Level Software Design Specification (LLD) provides the implementation-level design for the UME Organization Operating System. It describes every significant component, data structure, algorithm, and interface in sufficient detail for a Rust developer to implement the system without needing additional design decisions.

UME is structured as a layered Cargo workspace. The ume_core crate defines all shared contracts and domain types. ume_kernel provides the composition root and all subsystems. 42 organization modules in ume_modules implement the business domains. ume_server exposes the HTTP API. ume_runtime provides the CLI. The Java client suite provides GUI, shell, and watch modes built on Bazel.

Document Coverage

Section 1 — System Overview & Design Philosophy (8 architecture principles, runtime model, design patterns)
Section 2 — Crate Architecture (workspace layout, ume_core types, UmeError taxonomy, KernelModule trait, KernelFacade)
Section 3 — Kernel Core (UmeKernel composition root, KernelConfig field reference, 14-step boot sequence)
Section 4 — Kernel Subsystems (RBAC engine, event bus, StorageManager, supervisor, audit chain, executor pools, memory regions)
Section 5 — Module Architecture (standard module structure, repository pattern, permission declarations)
Section 6 — Key Module Designs (Finance, HR, Chombo, Risk, Process/Workflow, Soko/Marketing, Security, Work)
Section 7 — API Layer (ume_server Axum router, auth middleware, handler pattern, response envelopes, WebSocket, CLI)
Section 8 — Data Architecture (schema namespaces, multi-tenancy, schema conventions, migrations, event persistence, search)
Section 9 — Security Architecture (JWT design, crypto vault interface, input validation tiers, SQL injection prevention)
Section 10 — Device Driver Design (driver trait, storage/identity/comms/broker drivers, circuit breaker)
Section 11 — Deployment & Java Client (Docker image, Helm chart, env variables, Java client arch, SDK scaffold)
Appendix A — Complete Event Catalogue (35 domain events with payloads and subscribers)
Appendix B — Database Index Reference (all critical query path indexes)
Appendix C — Module Dependency Graph
Appendix D — Glossary

Key Design Decisions

- Single process kernel — all modules run in-process; no microservice network overhead or distributed coordination
- Trait-based abstraction everywhere — StorageDriver, IdentityDriver, CommDriver, CryptoVault are all swappable via config
- No direct module-to-module calls — all cross-module communication via event bus topics; modules are independently stoppable
- RBAC enforced in kernel facade — modules call assert_can(); they cannot bypass authorization
- Immutable audit chain — SHA-256 chained records; any deletion or modification is cryptographically detectable
- Newtype IDs everywhere — OrgUnitId(Uuid), EmployeeId(Uuid) etc. — compile-time prevention of wrong-ID bugs

- Multi-tenancy via org_id column — enforced unconditionally in StorageManager; no per-query opt-in required
- Three-tier input validation — serde macros → domain service validate() → database constraints

1. System Overview & Design Philosophy

1.1 Architecture Principles

UME is designed as a kernel-mediated organization operating system. Every architectural decision flows from eight foundational principles:

Principle	Statement	Consequence
Kernel Mediation	No module accesses any resource directly; all I/O flows through a kernel facade	Replaceability of all backends; uniform audit and security enforcement
Domain Isolation	Domain crates depend only on ume_core contracts; never on the kernel crate or other domain crates	No circular dependencies; modules are independently testable
Event-Driven Integration	Modules communicate via domain events on the event bus; no direct module-to-module function calls	Loose coupling; modules may be added, removed, or replaced without changing other modules
RBAC-First Security	Authorization is a kernel concern; modules declare permissions and rely on the kernel to enforce them	Consistent security model; no authorization gaps or bypass paths
Fault Isolation	A module in Error or Stopped state does not affect other modules or kernel operations	Resilience; partial degradation is always preferred to total failure
Immutable Audit	Audit records are append-only and cryptographically chained; no record may be modified or deleted	Non-repudiation; compliance-grade audit trail from day one
Composable Modules	Modules are the unit of feature delivery; any set of modules may be active without dependency conflicts	Progressive capability; organizations activate only what they need
Schema-First Data	All cross-module domain types are defined in ume_core before any module implements them	Single schema definition prevents cross-module data inconsistency

1.2 Runtime Architecture

At runtime, UME is a single process containing the kernel and all active modules. External systems are accessed exclusively via registered device drivers. Multiple UME server instances may be deployed behind a load balancer for horizontal scaling.

Runtime Component Model

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ ume_server (HTTP API)      ume_runtime (CLI)      Java Client      │
│      │                    │                    │          │
│      └──────────────────────────────────────┴──────────── HTTP ──────────┘          │
│                                          │
│ ┌──────────────────────────────── UME Kernel ──────────────────────────────────┐    │
│ │ KernelCore │ RBAC │ EventBus │ Executors │ Audit │    │
│ │ Supervisor │ DeviceBus │ StorageManager │ Memory    │    │
```

```
  │ │                                   │   │
  │ │ ┌──────────────── Organization Modules (1–42) ────────────────┐ │   │
  │ │ │ Finance │ HR │ Legal │ GRC │ Risk │ Sales │ ...     │ │   │
  │ │ └──────────────────────────────────────────────────────────────┘ │
 │
 │
 └──────────────────────────────────────────────────────────────────────┘
 │
  │                                   │
  │ ┌──────────── Device Drivers ────────────────────────────────────┐ │
  │ │ StorageDriver │ IdentityDriver │ CommDriver │ Broker │ │
  │
 └──────────────────────────────────────────────────────────────────────
┘ │
  │                                   │
  │ ┌────── External Systems ────────────────────────────────────────┐ │
  │ │ PostgreSQL / SQLite │ Redis │ SMTP │ NATS │ S3     │ │
  │
 └──────────────────────────────────────────────────────────────────────
──┘ │

 └──────────────────────────────────────────────────────────────────────
──────────┘
```

1.3 Key Design Patterns

Pattern	Where Used	Purpose
Newtype Wrapper	All domain IDs (OrgUnitId, EmployeeId, etc.)	Type-safe ID handling; prevents passing wrong ID type to functions
Builder Pattern	KernelConfig, BackupJobBuilder, CampaignBuilder	Validates invariants before object construction; ergonomic APIs
State Machine	Module lifecycle, workflow instances, deal stages	Explicit state transitions with side effects; prevents invalid states
Event Sourcing	Finance journals, GRC audit, Board resolutions	Reconstructable state from immutable event log; strong audit trail
CQRS (lightweight)	Analytics module	Separate read models for reporting; write models for transactions
Repository Pattern	All module data access	Abstracts StorageManager behind domain-typed interfaces
Strategy Pattern	ExecutorPool task routing, RBAC evaluators	Pluggable algorithms selected at configuration time
Observer / Event Bus	All cross-module communication	Decoupled publish-subscribe; no module directly calls another
Circuit Breaker	Driver health checks, external API calls	Prevents cascading failures on external system degradation
Decorator / Facade	Kernel service facades exposed to modules	Modules see a simplified, pre-authorized view of kernel services

2. Crate Architecture & Workspace Structure

2.1 Workspace Layout

UME is organized as a Cargo workspace. The dependency graph is strictly layered — higher layers depend on lower layers; no upward dependencies are permitted.

Rust

```
ume/
├── Cargo.toml                    # workspace root
├── Cargo.lock
├── ume_core/                     # Layer 0: shared types, traits, errors
├── ume_kernel/                   # Layer 1: kernel implementation
│   ├── src/
│   │   ├── lib.rs
│   │   ├── kernel.rs             # UmeKernel composition root
│   │   ├── config.rs             # KernelConfig
│   │   ├── supervisor.rs         # SupervisorEngine
│   │   ├── device_bus.rs         # DeviceBus + DeviceDriver trait
│   │   ├── rbac.rs               # RbacEngine
│   │   ├── event_bus.rs          # EventBus
│   │   ├── executor.rs           # ExecutorPool
│   │   ├── memory.rs             # MemoryManager
│   │   ├── storage.rs            # StorageManager
│   │   └── audit.rs              # LogAuditManager
├── ume_modules/                  # Layer 2: organization modules
│   ├── ume_admin/                # MOD-01 Administration
│   ├── ume_analytics/            # MOD-02 Analytics
│   ├── ume_backup/               # MOD-03 Backup
│   ├── ume_board/                # MOD-04 Board
│   ├── ume_bizdev/               # MOD-05 Business Development
│   ├── ume_cms/                  # MOD-06 Content Management
│   ├── ume_comms/                # MOD-07 Communications
│   ├── ume_crm/                  # MOD-08 CRM
│   ├── ume_design/               # MOD-09 Design System
│   ├── ume_engineering/          # MOD-10 Engineering
│   ├── ume_esg/                  # MOD-11 ESG
│   ├── ume_enterprise/           # MOD-12 Enterprise Mgmt
│   ├── ume_chombo/               # MOD-13 Legal Entity (Chombo)
│   ├── ume_finance/              # MOD-14 Finance
│   ├── ume_grc/                  # MOD-15 GRC
│   ├── ume_hr/                   # MOD-16 HR
│   ├── ume_investment/           # MOD-17 Investment
│   ├── ume_it/                   # MOD-18 IT & Assets
│   ├── ume_knowledge/            # MOD-19 Knowledge
│   ├── ume_learning/             # MOD-20 Learning
│   ├── ume_strategy/             # MOD-21 Strategy
│   ├── ume_soko/                 # MOD-22 Marketing (Soko)
│   ├── ume_mdm/                  # MOD-23 MDM
│   ├── ume_facilities/           # MOD-24 Facilities
│   ├── ume_operations/           # MOD-25 Operations
│   ├── ume_portal/               # MOD-26 Portal
│   ├── ume_portfolio/            # MOD-27 Portfolio
│   ├── ume_pr/                   # MOD-28 PR & Branding
│   ├── ume_process/              # MOD-29 Process & Workflow
│   ├── ume_product/              # MOD-30 Product
│   ├── ume_production/           # MOD-31 Production
│   ├── ume_requirements/         # MOD-32 Requirements
│   ├── ume_risk/                 # MOD-33 Risk
│   ├── ume_sales/                # MOD-34 Sales
│   ├── ume_schedule/             # MOD-35 Schedule
```

```
│   ├── ume_security/             # MOD-36 Security
│   ├── ume_supplychain/          # MOD-37 Supply Chain
│   ├── ume_teams/                # MOD-38 Teams
│   ├── ume_templates/            # MOD-39 Templates
│   └── ume_work/                 # MOD-40 Work Management
├── ume_server/                   # Layer 3: HTTP API server
├── ume_runtime/                  # Layer 3: CLI runtime
├── ume_sdk/                      # Layer 3: Custom module SDK
├── clients/
│   └── java/                     # Java client suite (Bazel)
├── tools/
│   ├── generate_custom_module/   # Module scaffold generator
│   └── migrate/                  # Schema migration runner
└── tests/                        # Integration test suite
```

2.2 ume_core — The Shared Foundation (Layer 0)

ume_core is the single most important crate in the workspace. It defines every cross-module contract. All other crates depend on it; it depends on nothing inside the workspace.

Rust

```rust
// ume_core/src/lib.rs
pub mod types;        // Newtype IDs, Money, DateRange, Pagination
pub mod traits;       // KernelModule, DeviceDriver, Repository<T>
pub mod errors;       // UmeError enum (exhaustive)
pub mod events;       // DomainEvent, EventEnvelope, EventMetadata
pub mod audit;        // AuditRecord, AuditOutcome
pub mod schema;       // All cross-module domain type definitions
pub mod facade;       // KernelFacade trait — what modules see of the kernel
pub mod rbac;         // Permission, Role, Subject types
pub mod config;       // ModuleConfig, ConfigValue, ConfigDescriptor
pub mod pagination;   // CursorPage<T>, PageRequest, PageResponse<T>
pub mod search;       // SearchQuery, SearchResult<T>
```

Rust

```rust
// ume_core/src/types.rs — newtype ID wrappers (sample)

use uuid::Uuid;
use rust_decimal::Decimal;
use chrono::{DateTime, Utc, NaiveDate};

macro_rules! define_id {
    ($name:ident) => {
        #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize)]
        pub struct $name(pub Uuid);
        impl $name {
            pub fn new() -> Self { Self(Uuid::new_v4()) }
            pub fn inner(&self) -> Uuid { self.0 }
        }
        impl Default for $name { fn default() -> Self { Self::new() } }
    };
}

define_id!(OrgId);            define_id!(OrgUnitId);
define_id!(EmployeeId);       define_id!(EntityId);
define_id!(JournalEntryId);   define_id!(InvoiceId);
```

```
define_id!(RiskId);          define_id!(WorkItemId);
define_id!(EventId);         define_id!(AuditRecordId);
// ... 40+ additional ID types

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Money {
    pub amount: Decimal,
    pub currency: CurrencyCode,  // ISO 4217
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct DateRange {
    pub start: NaiveDate,
    pub end: Option<NaiveDate>,
}
```

2.3 UmeError — Exhaustive Error Taxonomy

All fallible operations in UME return Result<T, UmeError>. The UmeError enum is defined in ume_core and covers every error class in the system.

Rust

```
// ume_core/src/errors.rs

#[derive(Debug, thiserror::Error)]
pub enum UmeError {
    // ── Authorization ──────────────────────────────────────────────
    #[error("Authorization denied: subject={subject} permission={permission}")]
    AuthorizationFailed { subject: String, permission: String },

    #[error("Authentication required")]
    Unauthenticated,

    // ── Not Found ──────────────────────────────────────────────────
    #[error("Resource not found: {resource_type} id={id}")]
    NotFound { resource_type: String, id: String },

    // ── Validation ─────────────────────────────────────────────────
    #[error("Validation failed: {errors:?}")]
    ValidationFailed { errors: Vec<FieldError> },

    #[error("Optimistic lock conflict: expected version={expected}
actual={actual}")]
    OptimisticLockConflict { expected: u64, actual: u64 },

    // ── Business Rules ─────────────────────────────────────────────
    #[error("Business rule violation: {rule} — {message}")]
    BusinessRuleViolation { rule: String, message: String },

    #[error("Period is locked: {period}")]
    PeriodLocked { period: String },

    // ── Infrastructure ─────────────────────────────────────────────
    #[error("Driver unavailable: {driver_id}")]
    DriverUnavailable { driver_id: String },

    #[error("Storage error: {source}")]
```

```
    StorageError { source: Box<dyn std::error::Error + Send + Sync> },

    #[error("Memory region exhausted: {region}")]
    MemoryRegionExhausted { region: String },

    #[error("Event bus capacity exceeded: {topic}")]
    BusCapacityExceeded { topic: String },

    // ── Module ──────────────────────────────────────────────────────
    #[error("Module not found: {module_id}")]
    ModuleNotFound { module_id: String },

    #[error("Module dependency unsatisfied: {module_id} requires {dependency}")]
    DependencyUnsatisfied { module_id: String, dependency: String },

    // ── Concurrency ─────────────────────────────────────────────────
    #[error("Operation timeout after {ms}ms")]
    Timeout { ms: u64 },

    // ── Domain-specific ─────────────────────────────────────────────
    #[error("Insufficient funds in {fund}: required={required}
available={available}")]
    InsufficientFunds { fund: String, required: String, available: String },

    #[error("RBAC cycle detected in role hierarchy")]
    RbacCycleDetected,

    #[error("Internal error: {message}")]
    Internal { message: String },
}
```

2.4 KernelModule Trait — Full Specification

Every organization module implements KernelModule. This trait is the complete contract between the kernel and all modules.

Rust

```
// ume_core/src/traits.rs

#[async_trait::async_trait]
pub trait KernelModule: Send + Sync + std::fmt::Debug {

    /// Unique, globally-scoped module identifier.
    /// Format: "{domain}.{name}" e.g. "finance.ledger", "hr.people"
    fn id(&self) -> &'static str;

    /// Domain area this module belongs to.
    fn domain(&self) -> Domain;

    /// Semantic version of this module implementation.
    fn version(&self) -> &'static str;

    /// Current lifecycle state of the module.
    fn state(&self) -> ModuleState;

    /// Start the module. Receives a KernelHandle providing access to all
```

```
    /// kernel services. Called after all declared dependencies are Running.
    async fn start(&mut self, kernel: Arc<dyn KernelFacade>) -> Result<(),
UmeError>;

    /// Stop the module gracefully. Must complete within drain_timeout.
    /// Release all kernel resources and complete or abort in-flight operations.
    async fn stop(&mut self) -> Result<(), UmeError>;

    /// Return current health. Called every supervisor_interval (default 30s).
    /// Must return within 500ms or supervisor records a health check timeout.
    async fn health_check(&self) -> HealthStatus;

    /// Return this module's descriptor: deps, permissions, config schema.
    fn describe(&self) -> ModuleDescriptor;

    /// Handle an inbound domain event from the event bus.
    /// Only called for topics declared in ModuleDescriptor::subscriptions.
    async fn on_event(&mut self, event: EventEnvelope) -> Result<(), UmeError>;

    /// Declare module dependencies: module IDs that must be Running first.
    fn dependencies(&self) -> Vec<&'static str>;

    /// Apply runtime configuration update.
    /// Returns true if a restart is required; false if hot-reload succeeded.
    async fn configure(&mut self, config: ModuleConfig) -> Result<bool, UmeError>;

    /// Optional: expose module-level Prometheus metrics.
    fn metrics(&self) -> Option<Box<dyn MetricsCollector>> { None }
}
```

Rust

```
// ume_core/src/traits.rs — ModuleDescriptor

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct ModuleDescriptor {
    pub id: &'static str,
    pub domain: Domain,
    pub version: &'static str,
    pub description: &'static str,
    pub dependencies: Vec<&'static str>,
    pub permissions: Vec<Permission>,      // permissions this module registers
    pub subscriptions: Vec<TopicPattern>,  // event bus topics this module
subscribes to
    pub config_schema: Vec<ConfigDescriptor>,
    pub capabilities: Vec<&'static str>,   // discoverable capability tags
}

#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub enum ModuleState {
    Registered,    // registered but not yet starting
    Initializing,  // start() called; awaiting dependency resolution
    Starting,      // dependencies met; executing start()
    Running,       // healthy and serving requests
    Degraded,      // health_check returned Warning; still serving
    Error,         // health_check failed; supervisor applying restart policy
    Stopping,      // stop() called; draining
    Stopped,       // cleanly stopped; may be restarted
}
```

2.5 KernelFacade — What Modules See

Modules receive a KernelFacade reference on start(). This is the only interface through which modules interact with the kernel. It is intentionally narrow — modules cannot access kernel internals.

Rust

```rust
// ume_core/src/facade.rs

#[async_trait::async_trait]
pub trait KernelFacade: Send + Sync {
    // ── Event Bus ──────────────────────────────────────────────────────
    async fn publish(&self, topic: &str, payload: serde_json::Value,
                     meta: EventMetadata) -> Result<EventId, UmeError>;
    async fn subscribe(&self, pattern: &str, handler: Arc<dyn EventHandler>)
                     -> Result<SubscriptionHandle, UmeError>;

    // ── Storage ────────────────────────────────────────────────────────
    async fn storage(&self) -> Arc<dyn StorageManager>;

    // ── RBAC ───────────────────────────────────────────────────────────
    async fn can(&self, subject: &SubjectId, permission: &str,
                 resource_id: Option<&str>) -> Result<bool, UmeError>;
    async fn assert_can(&self, subject: &SubjectId, permission: &str,
                        resource_id: Option<&str>) -> Result<(), UmeError>;

    // ── Audit ──────────────────────────────────────────────────────────
    async fn audit(&self, record: AuditRecord) -> Result<AuditRecordId, UmeError>;

    // ── Module Registry ────────────────────────────────────────────────
    fn module_by_id(&self, id: &str) -> Option<Arc<dyn KernelModule>>;
    fn modules_by_domain(&self, domain: Domain) -> Vec<Arc<dyn KernelModule>>;

    // ── Configuration ──────────────────────────────────────────────────
    fn config(&self, module_id: &str) -> Option<ModuleConfig>;

    // ── Scheduling ─────────────────────────────────────────────────────
    async fn schedule(&self, job: ScheduledJob) -> Result<JobHandle, UmeError>;
    async fn cancel_job(&self, handle: JobHandle) -> Result<(), UmeError>;
}
```

3. Kernel Core Design

3.1 UmeKernel — Composition Root

UmeKernel is the single composition root. It owns all subsystems and manages the complete lifecycle from boot to shutdown. It is constructed once per process via UmeKernel::boot(config).

Rust

```rust
// ume_kernel/src/kernel.rs

pub struct UmeKernel {
    config:        KernelConfig,
    state:         Arc<RwLock<KernelState>>,
    device_bus:    Arc<DeviceBus>,
    rbac:          Arc<RbacEngine>,
    event_bus:     Arc<EventBus>,
    executor_pool: Arc<ExecutorPool>,
    memory:        Arc<MemoryManager>,
    storage:       Arc<StorageManager>,
    audit:         Arc<LogAuditManager>,
    supervisor:    Arc<SupervisorEngine>,
    registry:      Arc<ModuleRegistry>,
    scheduler:     Arc<JobScheduler>,
    boot_time:     DateTime<Utc>,
}

impl UmeKernel {
    /// Primary boot sequence. Blocks until kernel is Running or returns error.
    pub async fn boot(config: KernelConfig) -> Result<Arc<Self>, UmeError> {
        tracing::info!("UME kernel booting — version {}", VERSION);

        // Phase 1: validate configuration
        config.validate()?;

        // Phase 2: initialize subsystems (order is critical)
        let memory    = MemoryManager::new(&config.memory)?;
        let audit     = LogAuditManager::new(&config.audit, memory.clone()).await?;
        let device_bus = DeviceBus::new(&config.drivers, audit.clone()).await?;
        let storage   = StorageManager::new(&config.storage,
device_bus.clone()).await?;
        let rbac      = RbacEngine::new(&config.rbac, storage.clone()).await?;
        let event_bus = EventBus::new(&config.events).await?;
        let executor_pool = ExecutorPool::new(&config.executors)?;
        let scheduler = JobScheduler::new(executor_pool.clone()).await?;

        // Phase 3: run schema migrations
        storage.run_migrations().await?;

        // Phase 4: construct registry and facade
        let registry  = ModuleRegistry::new();
        let kernel    = Arc::new(Self { config, state: /* Running */
                          device_bus, rbac, event_bus, executor_pool,
                          memory, storage, audit, scheduler,
                          supervisor: Arc::new(SupervisorEngine::new()),
                          registry, boot_time: Utc::now() });

        // Phase 5: start supervisor
        kernel.supervisor.start(kernel.clone()).await?;
```

```
        // Phase 6: register and start modules (dependency-ordered)
        kernel.start_modules().await?;

        tracing::info!("UME kernel ready in {:?}", kernel.boot_time.elapsed());
        Ok(kernel)
    }

    pub async fn shutdown(&self) -> Result<(), UmeError> {
        self.set_state(KernelState::ShuttingDown);
        self.registry.stop_all_modules().await?;  // reverse dependency order
        self.scheduler.shutdown().await?;
        self.event_bus.flush_and_close().await?;
        self.storage.flush_and_close().await?;
        self.device_bus.shutdown_all().await?;
        self.set_state(KernelState::Stopped);
        Ok(())
    }
}
```

3.2 KernelConfig — Full Field Specification

KernelConfig is deserialized from TOML/YAML/environment variables. All fields have documented defaults.

Field	Type	Nullable	Description
org_id	OrgId	No	Owning organization UUID — must match persisted value on warm restart
log_level	LogLevel	No	Default: INFO. Options: TRACE,DEBUG,INFO,WARN,ERROR
boot_timeout_secs	u64	No	Default: 30. Kernel boot fails if not Running within this window
supervisor.interval_secs	u64	No	Default: 30. Health check frequency for all modules
supervisor.max_restarts	u32	No	Default: 3. Max consecutive restarts before Stopped
supervisor.backoff_base_ms	u64	No	Default: 1000. Exponential backoff base in milliseconds
rbac.cache_ttl_secs	u64	No	Default: 60. RBAC decision cache TTL
rbac.enable_resource_scoping	bool	No	Default: true. Enable resource-level permission scoping
events.backend	EventBackend	No	Default: InMemory. Options: InMemory, Nats, Kafka, Rabbit
events.max_queue_depth	usize	No	Default: 10000. Per-topic queue depth before backpressure
events.dlq_retention_days	u32	No	Default: 30. Days to retain DLQ entries
storage.primary	StorageBackend	No	Default: Sqlite. Options: Sqlite, Postgres, InMemory
storage.dsn	String	Yes	Database connection string; required for Postgres

storage.pool_size	u32	No	Default: 10 for Postgres; N/A for Sqlite
storage.redis_url	String	Yes	Redis URL for hot cache tier; optional
memory.total_mb	u64	No	Default: 512. Total managed memory budget in MB
memory.regions	Vec<RegionConfig>	No	Per-region size limits and eviction policies
audit.retention_years	u32	No	Default: 7. Audit record retention in years
audit.chain_algorithm	HashAlg	No	Default: SHA256. Audit record chaining hash algorithm
modules.enabled	Vec<String>	No	List of module IDs to activate; empty = all built-ins
modules.dir	PathBuf	Yes	Directory to scan for custom module .so files
server.port	u16	No	Default: 8080. HTTP API listen port
server.tls_cert_path	PathBuf	Yes	TLS certificate PEM path; required for TLS mode
server.tls_key_path	PathBuf	Yes	TLS private key PEM path; required for TLS mode
server.rate_limit_rpm	u32	No	Default: 1000. Requests per minute per API client

3.3 Boot Sequence — Step-by-Step

Step	Phase	Action	Failure Behaviour
1	Config Validation	Parse and validate KernelConfig; check required fields and value ranges	Boot aborts; structured error message to stderr
2	Memory Init	Allocate named memory regions per configuration; register eviction policies	Boot aborts if region allocation fails
3	Audit Init	Initialize audit log writer; verify storage target writable; load chain seed	Boot aborts; audit is non-optional
4	Device Bus Init	Construct DeviceBus; register all configured drivers; call initialize() on each	Driver init failure: logged as WARNING; kernel continues with degraded driver set
5	Storage Init	Connect to primary storage backend via StorageDriver; test connection	Boot aborts; storage is non-optional
6	RBAC Init	Load role and permission definitions from storage; build evaluation graph	Boot aborts; RBAC is non-optional
7	Event Bus Init	Start in-process bus or connect to configured external broker	Boot aborts; event bus is non-optional
8	Executor Pool Init	Create named thread pools per config; verify thread spawn succeeds	Boot aborts if critical executors cannot start
9	Scheduler Init	Start cron job scheduler bound to executor pool	Boot aborts if scheduler fails

10	Schema Migration	Run pending migrations in order; record each migration in migration_log	Boot aborts if migration fails; migration is idempotent
11	Module Registration	Register all enabled modules in registry; build dependency graph	Boot aborts if dependency cycle detected
12	Module Start	Start modules in dependency-topological order; wait for Running state	Module start failure: supervisor records; kernel continues if non-critical
13	Supervisor Start	Start background health check loop at configured interval	Boot aborts if supervisor fails to start
14	Ready Signal	Emit kernel.ready event; update state to Running; log boot duration	—

4. Kernel Subsystems — Detailed Design

4.1 RBAC Engine

The RBAC engine is the sole authorization decision point for the entire system. It implements a hierarchical, resource-scoped permission model.

Rust

```rust
// ume_kernel/src/rbac.rs — core types

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Permission {
    pub id: &'static str,          // e.g. "finance.journal.create"
    pub description: &'static str,
    pub resource_scoped: bool,     // true = can be scoped to specific resource ID
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Role {
    pub id: RoleId,
    pub name: String,
    pub description: String,
    pub permissions: Vec<PermissionGrant>,
    pub parent_roles: Vec<RoleId>,  // inheritance
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct PermissionGrant {
    pub permission: String,
    pub resource_id: Option<String>,  // None = all resources
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Subject {
    pub id: SubjectId,
    pub subject_type: SubjectType,  // User | ServiceAccount | ApiKey
    pub org_id: OrgId,
    pub roles: Vec<RoleId>,
}

impl RbacEngine {
    /// Core authorization check with caching.
    pub async fn can(&self, subject_id: &SubjectId, permission: &str,
                     resource_id: Option<&str>) -> Result<bool, UmeError> {
        // 1. Cache lookup
        let cache_key = self.cache_key(subject_id, permission, resource_id);
        if let Some(result) = self.cache.get(&cache_key).await {
            return Ok(result);
        }

        // 2. Load subject roles (with inheritance expansion)
        let subject = self.subjects.get(subject_id).await?;
        let effective_permissions = self.expand_permissions(&subject).await?;

        // 3. Check permission (resource-scoped or global)
        let granted = effective_permissions.iter().any(|grant| {
            grant.permission == permission &&
            (grant.resource_id.is_none() ||
             grant.resource_id.as_deref() == resource_id)
        });
```

```
        // 4. Cache result
        self.cache.set(cache_key, granted, self.config.cache_ttl).await;

        Ok(granted)
    }

    /// Expand role hierarchy to full effective permission set.
    async fn expand_permissions(&self, subject: &Subject)
        -> Result<Vec<PermissionGrant>, UmeError>
    {
        let mut visited = HashSet::new();
        let mut permissions = Vec::new();
        for role_id in &subject.roles {
            self.collect_role_permissions(role_id, &mut permissions,
                                          &mut visited).await?;
        }
        Ok(permissions)
    }
}
```

4.2 Event Bus

The EventBus provides publish-subscribe semantics with backpressure, DLQ, and pluggable backends.

Rust

```
// ume_kernel/src/event_bus.rs

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct EventEnvelope {
    pub id:          EventId,
    pub topic:       String,          // {domain}.{resource}.{verb}
    pub payload:     serde_json::Value,
    pub published_at: DateTime<Utc>,
    pub publisher_module: String,
    pub correlation_id: Option<String>,
    pub causation_id:   Option<EventId>,  // ID of event that caused this one
    pub idempotency_key: Option<String>,
}

pub struct EventBus {
    backend:     Arc<dyn BusBackend>,  // InMemory | Nats | Kafka | Rabbit
    dlq:         Arc<DeadLetterQueue>,
    topic_regex: Regex,               // validates topic format
}

impl EventBus {
    /// Publish an event. Validates topic format; applies backpressure if queue
full.
    pub async fn publish(&self, envelope: EventEnvelope) -> Result<(), UmeError> {
        // Validate topic format: {domain}.{resource}.{verb}
        if !self.topic_regex.is_match(&envelope.topic) {
            return Err(UmeError::ValidationFailed {
                errors: vec![FieldError::new("topic", "must match
{domain}.{resource}.{verb}")]
            });
        }
        // Idempotency deduplication
```

```
        if let Some(key) = &envelope.idempotency_key {
            if self.backend.is_duplicate(key).await? { return Ok(()); }
        }
        self.backend.publish(envelope).await
    }

    /// Subscribe to a topic pattern (glob supported: "finance.*", "*.created").
    pub async fn subscribe(&self, pattern: &str,
                           handler: Arc<dyn EventHandler>)
                           -> Result<SubscriptionHandle, UmeError> {
        self.backend.subscribe(pattern, handler).await
    }
}

pub trait EventHandler: Send + Sync {
    async fn handle(&self, envelope: EventEnvelope) -> Result<(), UmeError>;
    fn on_failure(&self, envelope: EventEnvelope, err: UmeError) {
        // Default: route to DLQ
    }
}
```

4.3 StorageManager

The StorageManager abstracts all persistence. Modules call storage operations; they never interact with the database directly.

Rust

```
// ume_kernel/src/storage.rs

#[async_trait]
pub trait StorageManager: Send + Sync {
    // ── Basic CRUD ──────────────────────────────────────────────────────
    async fn get<T: DeserializeOwned + Send>(
        &self, namespace: &str, id: &str) -> Result<Option<T>, UmeError>;

    async fn put<T: Serialize + Send>(
        &self, namespace: &str, id: &str, value: &T,
        version: Option<u64>  // optimistic lock; None = unconditional
    ) -> Result<u64, UmeError>;  // returns new version

    async fn delete(
        &self, namespace: &str, id: &str) -> Result<(), UmeError>;

    // ── Query ───────────────────────────────────────────────────────────
    async fn query<T: DeserializeOwned + Send>(
        &self, namespace: &str, filter: Filter,
        page: PageRequest) -> Result<PageResponse<T>, UmeError>;

    // ── Transactions ────────────────────────────────────────────────────
    async fn transaction<F, T>(&self, f: F) -> Result<T, UmeError>
    where
        F: FnOnce(Arc<dyn TransactionContext>) -> BoxFuture<'_, Result<T,
UmeError>>
        + Send;

    // ── Schema ──────────────────────────────────────────────────────────
    async fn run_migrations(&self) -> Result<(), UmeError>;
```

```
    async fn migration_status(&self) -> Result<Vec<MigrationRecord>, UmeError>;
}

// Storage tier routing — configured per namespace
pub struct TieredStorageManager {
    hot_cache:  Option<Arc<dyn StorageBackend>>,  // Redis
    warm_store: Arc<dyn StorageBackend>,          // Postgres/Sqlite
    cold_store: Option<Arc<dyn StorageBackend>>,  // S3 for archives
    config:     StorageRoutingConfig,
}
```

4.4 Supervisor Engine

The supervisor monitors all modules and applies restart policies. It runs as a background task in the kernel.critical executor.

Rust

```
// ume_kernel/src/supervisor.rs

pub struct SupervisorEngine {
    restart_counts: Arc<DashMap<String, (u32, DateTime<Utc>)>>,
    config: SupervisorConfig,
}

impl SupervisorEngine {
    pub async fn run_loop(&self, kernel: Arc<UmeKernel>) {
        let mut interval = tokio::time::interval(
            Duration::from_secs(self.config.interval_secs));

        loop {
            interval.tick().await;
            let modules = kernel.registry.all_running().await;

            for module in modules {
                let health = tokio::time::timeout(
                    Duration::from_millis(500),
                    module.health_check()
                ).await;

                match health {
                    Ok(HealthStatus::Healthy) => {
                        self.reset_restart_count(module.id());
                    }
                    Ok(HealthStatus::Warning(msg)) => {
                        tracing::warn!("Module {} degraded: {}", module.id(), msg);
                        kernel.registry.set_state(module.id(),
ModuleState::Degraded);
                    }
                    Ok(HealthStatus::Unhealthy(msg)) | Err(_) => {
                        self.handle_failure(&kernel, &module, &msg).await;
                    }
                }
            }
        }
    }

    async fn handle_failure(&self, kernel: &Arc<UmeKernel>,
                            module: &Arc<dyn KernelModule>, msg: &str) {
```

```
        let (count, last) = self.restart_counts
            .entry(module.id().to_string())
            .or_insert((0, Utc::now()));

        *count += 1;
        if *count > self.config.max_restarts {
            tracing::error!("Module {} exceeded max restarts; stopping",
module.id());
            kernel.registry.stop_module(module.id()).await;
            kernel.publish_alert(Alert::module_stopped(module.id(), msg)).await;
            return;
        }

        // Exponential backoff
        let backoff = Duration::from_millis(
            self.config.backoff_base_ms * 2u64.pow(*count - 1));
        tracing::warn!("Module {} failed (attempt {}); restarting in {:?}",
            module.id(), count, backoff);
        tokio::time::sleep(backoff).await;
        kernel.registry.restart_module(module.id()).await;
    }
}
```

4.5 Log & Audit Manager

The audit manager writes immutable, cryptographically-chained audit records. Chain integrity ensures any tampering with historical records is detectable.

Rust

```
// ume_kernel/src/audit.rs

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct AuditRecord {
    pub id:           AuditRecordId,            // UUID v4
    pub at:           DateTime<Utc>,            // RFC3339 UTC
    pub actor:        SubjectId,                // who performed the action
    pub actor_type:   SubjectType,              // User | Service | Scheduler
    pub action:       String,                   // {domain}.{resource}.{verb}
    pub resource_id:  Option<String>,           // affected resource UUID
    pub resource_type: String,                  // e.g. "finance.JournalEntry"
    pub outcome:      AuditOutcome,             // Success | Failure | Denied
    pub detail:       serde_json::Value,        // action-specific detail
    pub session_id:   Option<String>,           // HTTP session or CLI session
    pub request_id:   Option<String>,           // correlation ID from API request
    pub ip_address:   Option<std::net::IpAddr>,
    pub org_id:       OrgId,                    // always set; enables multi-tenant
isolation
    pub previous_hash: Option<String>,          // SHA-256 of previous record
    pub self_hash:     String,                  // SHA-256 of this record (minus
self_hash)
}

impl LogAuditManager {
    pub async fn write(&self, record: AuditRecord) -> Result<AuditRecordId,
UmeError> {
        // 1. Fetch previous record hash (atomic read)
        let prev_hash = self.chain_tail.read().await.clone();
```

```
        // 2. Set chain link
        let mut record = record;
        record.previous_hash = prev_hash;

        // 3. Compute self hash (excludes self_hash field)
        record.self_hash = self.compute_hash(&record);

        // 4. Persist (append-only; no UPDATE path exists)
        let id = self.storage.append_audit(&record).await?;

        // 5. Update chain tail
        *self.chain_tail.write().await = Some(record.self_hash.clone());

        // 6. Publish to event bus for SIEM integration
        self.event_bus.publish(EventEnvelope {
            topic: "audit.record.created".to_string(),
            payload: serde_json::to_value(&record).unwrap(),
            ..Default::default()
        }).await?;

        Ok(id)
    }

    /// Verify chain integrity from start_id to end_id.
    pub async fn verify_chain(&self, start_id: &AuditRecordId,
                              end_id: &AuditRecordId)
                              -> Result<ChainVerificationResult, UmeError> {
        let records = self.storage
            .query_audit_range(start_id, end_id).await?;

        for window in records.windows(2) {
            let (prev, curr) = (&window[0], &window[1]);
            let expected = self.compute_hash(prev);
            if curr.previous_hash.as_deref() != Some(&expected) {
                return Ok(ChainVerificationResult::Broken {
                    at_record: curr.id,
                });
            }
        }
        Ok(ChainVerificationResult::Valid)
    }
}
```

4.6 ExecutorPool

Named executor pools prevent any single domain from monopolizing CPU resources. Each pool is a Tokio runtime handle with bounded channel capacity.

Pool Name	Default Threads	Queue Depth	Purpose
kernel.critical	4	Unbounded	Supervisor, config changes, kernel lifecycle
kernel.events	8	10000	Event bus delivery to module handlers
kernel.audit	2	50000	Audit record writes (high volume, must not block)
org.finance	4	5000	Finance module: ledger, payroll, invoicing
org.hr	2	2000	HR module: payroll compute, review cycles

org.analytics	4	1000	Analytics KPI computation; longer-running queries
org.ops	4	5000	Operations, supply chain, work management
org.background	2	20000	Low-priority jobs: report generation, cleanup tasks
org.comms	2	10000	Email, SMS, notification dispatch

4.7 MemoryManager

The MemoryManager provides named, bounded memory regions. Each region has its own allocation limit and eviction policy.

Region	Default Size	Eviction Policy	Primary Consumer
kernel.hot_cache	64 MB	LRU (by access time)	RBAC decision cache; module lookup cache
kernel.event_queue	32 MB	Backpressure (reject new)	In-memory event bus queues
kernel.audit_buffer	16 MB	Threshold flush (80%)	Buffered audit record batch writes
module.analytics	128 MB	TTL (60 min) + LRU	Computed KPI aggregates; dashboard data
module.search_index	64 MB	LRU (by access time)	Full-text search index cache
module.templates	16 MB	TTL (10 min)	Compiled template cache
module.session	32 MB	TTL (session expiry)	HTTP session state; JWT nonce cache
module.general	Remainder	LRU (by access time)	All other module working data

5. Organization Module Architecture

5.1 Standard Module Structure

Every module follows an identical internal structure. This uniformity simplifies maintenance, testing, and code review.

Rust

```
ume_finance/                          # typical module crate
├── src/
│   ├── lib.rs                        # crate root; exports FinanceModule
│   ├── module.rs                     # KernelModule impl (start/stop/health/etc.)
│   ├── config.rs                     # FinanceConfig; ConfigDescriptor
declarations
│   ├── domain/                       # domain types (re-export from ume_core)
│   │   ├── mod.rs
│   │   ├── journal.rs                # JournalEntry, LedgerAccount types
│   │   ├── invoice.rs                # Invoice, InvoiceLine types
│   │   └── budget.rs                 # BudgetLine, VarianceAlert types
│   ├── repository/                   # storage abstraction layer
│   │   ├── mod.rs
│   │   ├── journal_repo.rs           # JournalRepository trait + impl
│   │   └── invoice_repo.rs           # InvoiceRepository trait + impl
│   ├── services/                     # business logic layer
│   │   ├── ledger_service.rs         # double-entry posting logic
│   │   ├── payroll_service.rs        # payroll compute
│   │   └── consolidation_service.rs  # multi-entity consolidation
│   ├── handlers/                     # event bus handlers
│   │   ├── hr_events.rs              # handles hr.employee.hired → create payroll
record
│   │   └── supplychain_events.rs     # handles sc.po.received → post GR journal
│   └── permissions.rs               # FINANCE_PERMISSIONS constant array
└── tests/
    ├── journal_tests.rs
    └── integration_tests.rs          # uses MockKernel from ume_sdk
```

5.2 Repository Pattern

Every module wraps StorageManager calls in typed Repository structs. This provides a domain-typed interface and isolates the storage API from business logic.

Rust

```
// Pattern: Repository trait + implementation

#[async_trait]
pub trait JournalRepository: Send + Sync {
    async fn create(&self, entry: &JournalEntry) -> Result<JournalEntryId,
UmeError>;
    async fn get(&self, id: &JournalEntryId) -> Result<Option<JournalEntry>,
UmeError>;
    async fn list_by_period(&self, period: &AccountingPeriod,
                            page: PageRequest) ->
Result<PageResponse<JournalEntry>, UmeError>;
    async fn list_by_account(&self, account: &LedgerAccountCode,
                             page: PageRequest) ->
Result<PageResponse<JournalEntry>, UmeError>;
```

```
}

pub struct SqlJournalRepository {
    storage: Arc<dyn StorageManager>,
    org_id:  OrgId,
}

#[async_trait]
impl JournalRepository for SqlJournalRepository {
    async fn create(&self, entry: &JournalEntry) -> Result<JournalEntryId,
UmeError> {
        // All writes are org_id scoped — multi-tenancy enforced here
        let mut storable = entry.clone();
        storable.org_id = self.org_id;
        self.storage.put("finance.journal_entries",
                        &entry.id.to_string(), &storable, None).await?;
        Ok(entry.id)
    }
}
```

5.3 Module Permission Declarations

Each module declares its RBAC permission namespace as a static array. The kernel registers all permissions at module load time.

Rust

```
// ume_finance/src/permissions.rs

pub const FINANCE_PERMISSIONS: &[Permission] = &[
    Permission { id: "finance.journal.create",
                 description: "Post journal entries to the general ledger",
                 resource_scoped: false },
    Permission { id: "finance.journal.view",
                 description: "View journal entries",
                 resource_scoped: true },
    Permission { id: "finance.period.lock",
                 description: "Lock an accounting period",
                 resource_scoped: false },
    Permission { id: "finance.invoice.create",
                 description: "Create customer or supplier invoices",
                 resource_scoped: false },
    Permission { id: "finance.invoice.approve",
                 description: "Approve invoices for payment",
                 resource_scoped: true },
    Permission { id: "finance.payment.authorize",
                 description: "Authorize payment runs",
                 resource_scoped: false },
    Permission { id: "finance.budget.manage",
                 description: "Create and modify budget lines",
                 resource_scoped: true },
    Permission { id: "finance.statements.view",
                 description: "View financial statements",
                 resource_scoped: false },
    Permission { id: "finance.admin",
                 description: "Full finance module administration",
                 resource_scoped: false },
];
```

6. Key Module Designs

6.1 Finance & Accounting Module (MOD-14)

6.1.1 Data Model

SQL / DDL

```
-- finance.ledger_accounts
CREATE TABLE finance.ledger_accounts (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id          UUID NOT NULL REFERENCES admin.organizations(id),
    code            VARCHAR(20) NOT NULL,
    name            VARCHAR(200) NOT NULL,
    account_type    VARCHAR(20) NOT NULL CHECK (account_type IN
                    ('Asset','Liability','Equity','Revenue','Expense')),
    parent_code     VARCHAR(20),
    is_active       BOOLEAN NOT NULL DEFAULT true,
    entity_id       UUID REFERENCES chombo.legal_entities(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    version         BIGINT NOT NULL DEFAULT 1,
    UNIQUE(org_id, entity_id, code)
);

-- finance.journal_entries (immutable after posting)
CREATE TABLE finance.journal_entries (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id          UUID NOT NULL,
    entity_id       UUID NOT NULL,
    period          VARCHAR(7) NOT NULL,  -- YYYY-MM
    entry_date      DATE NOT NULL,
    reference       VARCHAR(100),
    description     TEXT NOT NULL,
    entry_type      VARCHAR(30) NOT NULL, -- Manual|Payroll|AP|AR|Accrual|Reversal
    status          VARCHAR(20) NOT NULL DEFAULT 'Draft',
    posted_at       TIMESTAMPTZ,
    posted_by       UUID,
    source_module   VARCHAR(50),          -- originating module ID
    source_id       UUID,                 -- originating record (e.g. invoice_id)
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- finance.journal_lines (double-entry lines)
CREATE TABLE finance.journal_lines (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    entry_id        UUID NOT NULL REFERENCES finance.journal_entries(id),
    org_id          UUID NOT NULL,
    account_code    VARCHAR(20) NOT NULL,
    debit           NUMERIC(19,4),
    credit          NUMERIC(19,4),
    currency        CHAR(3) NOT NULL DEFAULT 'USD',
    fx_rate         NUMERIC(12,6) NOT NULL DEFAULT 1.0,
    functional_debit  NUMERIC(19,4),  -- converted to functional currency
    functional_credit NUMERIC(19,4),
    cost_center     VARCHAR(50),
    project_code    VARCHAR(50)
);

-- Constraint: every posted entry must balance (trigger-enforced)
CREATE OR REPLACE FUNCTION finance.check_journal_balance()
```

```
RETURNS TRIGGER AS $$
DECLARE total_debit NUMERIC; total_credit NUMERIC;
BEGIN
  SELECT COALESCE(SUM(debit),0), COALESCE(SUM(credit),0)
  INTO total_debit, total_credit
  FROM finance.journal_lines WHERE entry_id = NEW.id;
  IF total_debit != total_credit THEN
    RAISE EXCEPTION 'Journal entry % does not balance: Dr=% Cr=%',
      NEW.id, total_debit, total_credit;
  END IF; RETURN NEW;
END; $$ LANGUAGE plpgsql;
```

6.1.2 Ledger Service — Double-Entry Logic

Rust

```rust
// ume_finance/src/services/ledger_service.rs

pub struct LedgerService {
    journal_repo: Arc<dyn JournalRepository>,
    account_repo: Arc<dyn AccountRepository>,
    period_repo:  Arc<dyn PeriodRepository>,
    kernel:       Arc<dyn KernelFacade>,
}

impl LedgerService {
    /// Post a journal entry. Enforces period lock, balance, and permissions.
    pub async fn post_journal(&self, cmd: PostJournalCommand,
                              actor: &SubjectId) -> Result<JournalEntryId, UmeError>
{
        // 1. Authorization
        self.kernel.assert_can(actor, "finance.journal.create", None).await?;

        // 2. Period lock check
        let period = self.period_repo.get(&cmd.period).await?
            .ok_or(UmeError::NotFound { resource_type: "Period".into(),
                                        id: cmd.period.clone() })?;
        if period.is_locked {
            return Err(UmeError::PeriodLocked { period: cmd.period.clone() });
        }

        // 3. Balance validation (must sum to zero)
        let sum: Decimal = cmd.lines.iter()
            .map(|l| l.debit.unwrap_or_default() - l.credit.unwrap_or_default())
            .sum();
        if sum != Decimal::ZERO {
            return Err(UmeError::ValidationFailed {
                errors: vec![FieldError::new("lines",
                    &format!("Entry does not balance: net={}", sum))],
            });
        }

        // 4. Validate all accounts exist
        for line in &cmd.lines {
            self.account_repo.get(&line.account_code).await?
                .ok_or_else(|| UmeError::NotFound {
                    resource_type: "LedgerAccount".into(),
                    id: line.account_code.clone(),
                })?;
        }
```

```
        // 5. Persist within transaction
        let entry_id = self.kernel.storage().await
            .transaction(|tx| Box::pin(async move {
                let id = journal_repo.create_in_tx(&cmd.into_entry(), tx).await?;
                Ok(id)
            })).await?;

        // 6. Audit + event
        self.kernel.audit(AuditRecord::new(actor, "finance.journal.create",
            &entry_id.to_string(), AuditOutcome::Success)).await?;
        self.kernel.publish("finance.journal.posted",
            json!({ "entry_id": entry_id, "period": cmd.period }), /* meta */
).await?;

        Ok(entry_id)
    }
}
```

6.2 Human Resources Module (MOD-16)

6.2.1 Employee Data Model

SQL / DDL

```
-- hr.employees
CREATE TABLE hr.employees (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id          UUID NOT NULL,
    employee_number VARCHAR(30) UNIQUE,
    first_name      VARCHAR(100) NOT NULL,
    last_name       VARCHAR(100) NOT NULL,
    preferred_name  VARCHAR(100),
    email           VARCHAR(254) NOT NULL,
    phone           VARCHAR(30),
    status          VARCHAR(20) NOT NULL DEFAULT 'PreStart'
                    CHECK (status IN ('PreStart','Active','Leave',
                                      'Suspended','Terminated')),
    org_unit_id     UUID NOT NULL REFERENCES admin.org_units(id),
    manager_id      UUID REFERENCES hr.employees(id),
    employment_type VARCHAR(20) NOT NULL,  -- FullTime|PartTime|Contract|Casual
    start_date      DATE NOT NULL,
    end_date        DATE,
    probation_end   DATE,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    version         BIGINT NOT NULL DEFAULT 1
);

-- hr.compensation
CREATE TABLE hr.compensation (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    employee_id     UUID NOT NULL REFERENCES hr.employees(id),
    org_id          UUID NOT NULL,
    pay_type        VARCHAR(20) NOT NULL,  -- Salary|Hourly|Commission
    gross_amount    NUMERIC(12,2) NOT NULL,
    currency        CHAR(3) NOT NULL,
    pay_frequency   VARCHAR(20) NOT NULL,  -- Monthly|Biweekly|Weekly
    effective_from  DATE NOT NULL,
    effective_to    DATE,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

```
);

-- hr.leave_balances
CREATE TABLE hr.leave_balances (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    employee_id     UUID NOT NULL REFERENCES hr.employees(id),
    org_id          UUID NOT NULL,
    leave_type      VARCHAR(50) NOT NULL,
    balance_days    NUMERIC(6,2) NOT NULL DEFAULT 0,
    accrued_ytd     NUMERIC(6,2) NOT NULL DEFAULT 0,
    taken_ytd       NUMERIC(6,2) NOT NULL DEFAULT 0,
    year            INT NOT NULL,
    UNIQUE (employee_id, leave_type, year)
);
```

6.2.2 Onboarding Workflow Integration

When HR creates an employee with status PreStart, it emits hr.employee.hired. The Process Module receives this event and spawns a parameterized onboarding workflow instance.

Rust

```
// ume_hr/src/services/employee_service.rs (excerpt)

pub async fn create_employee(&self, cmd: CreateEmployeeCommand,
                             actor: &SubjectId) -> Result<EmployeeId, UmeError> {
    self.kernel.assert_can(actor, "hr.employee.create", None).await?;

    let employee = Employee::from_command(cmd);
    let id = self.employee_repo.create(&employee).await?;

    // Emit domain event — Process Module listens and starts onboarding
    self.kernel.publish("hr.employee.hired", json!({
        "employee_id": id,
        "org_unit_id": employee.org_unit_id,
        "start_date":  employee.start_date,
        "manager_id":  employee.manager_id,
    }), EventMetadata::from_actor(actor)).await?;

    self.kernel.audit(AuditRecord::new(actor, "hr.employee.create",
        &id.to_string(), AuditOutcome::Success)).await?;

    Ok(id)
}
```

6.3 Legal Entity Module — Chombo (MOD-13)

6.3.1 Entity & Policy Pack Data Model

SQL / DDL

```
-- chombo.legal_entities
CREATE TABLE chombo.legal_entities (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id            UUID NOT NULL,
    name              VARCHAR(300) NOT NULL,
```

```
    entity_type       VARCHAR(50) NOT NULL,
    jurisdiction      CHAR(2) NOT NULL,  -- ISO 3166-1 alpha-2
    sub_jurisdiction  VARCHAR(10),       -- state/province code
    registration_number VARCHAR(100),
    status            VARCHAR(20) NOT NULL DEFAULT 'Active'
                      CHECK (status IN ('Pending','Active','Dormant','Dissolved')),
    parent_entity_id  UUID REFERENCES chombo.legal_entities(id),
    ownership_pct     NUMERIC(5,2),
    incorporated_date DATE,
    fiscal_year_end   VARCHAR(5),  -- MM-DD
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    version           BIGINT NOT NULL DEFAULT 1
);

-- chombo.policy_packs
CREATE TABLE chombo.policy_packs (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    pack_id       VARCHAR(100) UNIQUE NOT NULL,
    jurisdiction  CHAR(2) NOT NULL,
    sub_jurisdiction VARCHAR(10),
    entity_types  TEXT[],  -- which entity types this pack applies to
    name          VARCHAR(200) NOT NULL,
    version       VARCHAR(20) NOT NULL,
    rules         JSONB NOT NULL,  -- serialized PolicyRule[]
    is_active     BOOLEAN NOT NULL DEFAULT true
);

-- chombo.filing_records
CREATE TABLE chombo.filing_records (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    entity_id     UUID NOT NULL REFERENCES chombo.legal_entities(id),
    org_id        UUID NOT NULL,
    filing_type   VARCHAR(100) NOT NULL,
    due_date      DATE NOT NULL,
    status        VARCHAR(20) NOT NULL DEFAULT 'Pending'
                  CHECK (status IN ('Pending','InProgress','Filed','Overdue')),
    filed_at      TIMESTAMPTZ,
    evidence_id   UUID,  -- CMS document reference
    regulator_ref VARCHAR(100)
);
```

6.3.2 Policy Pack Evaluation Engine

Rust

```rust
// ume_chombo/src/services/evaluation_service.rs

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct PolicyRule {
    pub rule_id:     String,
    pub name:        String,
    pub condition:   RuleCondition,  // evaluated against entity fields
    pub obligation:  ObligationType, // Filing | Registration | Disclosure |
Maintenance
    pub filing_type: Option<String>,
    pub lead_days:   Option<u32>,
    pub severity:    AlertSeverity,  // Critical | High | Medium | Low
}

pub struct EvaluationService {
```

```
    pack_repo:   Arc<dyn PolicyPackRepository>,
    entity_repo: Arc<dyn EntityRepository>,
    kernel:      Arc<dyn KernelFacade>,
}

impl EvaluationService {
    /// Evaluate an entity against all applicable policy packs.
    /// Returns compliance alerts and derived filing schedule.
    pub async fn evaluate(&self, entity_id: &EntityId)
            -> Result<EvaluationResult, UmeError> {
        let entity = self.entity_repo.get(entity_id).await?
            .ok_or_else(|| UmeError::not_found("LegalEntity", entity_id))?;

        // Load all packs applicable to this jurisdiction + entity type
        let packs = self.pack_repo
            .by_jurisdiction_and_type(&entity.jurisdiction, &entity.entity_type)
            .await?;

        // Evaluate all packs in parallel (rayon for CPU-bound work)
        let results: Vec<PackResult> = packs.par_iter()
            .map(|pack| self.evaluate_pack(&entity, pack))
            .collect();

        let alerts: Vec<ComplianceAlert> = results.iter()
            .flat_map(|r| r.alerts.clone()).collect();
        let filings: Vec<FilingScheduleEntry> = results.iter()
            .flat_map(|r| r.filings.clone()).collect();

        // Publish evaluation completed event
        self.kernel.publish("chombo.entity.evaluated", json!({
            "entity_id": entity_id,
            "alert_count": alerts.len(),
            "critical_alerts": alerts.iter().filter(|a| a.is_critical()).count(),
        }), Default::default()).await?;

        Ok(EvaluationResult { entity_id: *entity_id, alerts, filings })
    }

    fn evaluate_pack(&self, entity: &LegalEntity, pack: &PolicyPack) -> PackResult
{
        let mut alerts = Vec::new();
        let mut filings = Vec::new();

        for rule in &pack.rules {
            if rule.condition.matches(entity) {
                match &rule.obligation {
                    ObligationType::Filing => {
                        filings.push(FilingScheduleEntry::from_rule(rule, entity));
                    }
                    _ => {
                        alerts.push(ComplianceAlert::from_rule(rule, entity));
                    }
                }
            }
        }
        PackResult { pack_id: pack.pack_id.clone(), alerts, filings }
    }
}
```

6.4 Risk Management Module (MOD-33)

6.4.1 Risk Data Model

SQL / DDL

```
-- risk.risks
CREATE TABLE risk.risks (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id          UUID NOT NULL,
    risk_ref        VARCHAR(30) UNIQUE,     -- human-readable ref: RISK-2026-001
    title           VARCHAR(300) NOT NULL,
    description     TEXT NOT NULL,
    category        VARCHAR(50) NOT NULL,   -- Strategic|Financial|Operational|...
    sub_category    VARCHAR(50),
    inherent_likelihood  SMALLINT NOT NULL CHECK (inherent_likelihood BETWEEN 1 AND
5),
    inherent_impact      SMALLINT NOT NULL CHECK (inherent_impact BETWEEN 1 AND 5),
    inherent_score       SMALLINT GENERATED ALWAYS AS
                         (inherent_likelihood * inherent_impact) STORED,
    residual_likelihood  SMALLINT CHECK (residual_likelihood BETWEEN 1 AND 5),
    residual_impact      SMALLINT CHECK (residual_impact BETWEEN 1 AND 5),
    residual_score       SMALLINT GENERATED ALWAYS AS
                         (residual_likelihood * residual_impact) STORED,
    treatment_type  VARCHAR(20) NOT NULL DEFAULT 'Mitigate'
                    CHECK (treatment_type IN
('Accept','Mitigate','Transfer','Avoid')),
    owner_id        UUID REFERENCES hr.employees(id),
    sponsor_id      UUID REFERENCES hr.employees(id),
    status          VARCHAR(20) NOT NULL DEFAULT 'Open',
    appetite_breached BOOLEAN GENERATED ALWAYS AS (residual_score > 12) STORED,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    version         BIGINT NOT NULL DEFAULT 1
);

-- risk.kris
CREATE TABLE risk.kris (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    risk_id         UUID NOT NULL REFERENCES risk.risks(id),
    org_id          UUID NOT NULL,
    name            VARCHAR(200) NOT NULL,
    metric_source   VARCHAR(100),  -- analytics KPI code or manual
    current_value   NUMERIC(12,4),
    amber_threshold NUMERIC(12,4) NOT NULL,
    red_threshold   NUMERIC(12,4) NOT NULL,
    unit            VARCHAR(30),
    last_updated    TIMESTAMPTZ
);
```

6.4.2 KRI Monitoring

Rust

```
// ume_risk/src/services/kri_monitor.rs

pub struct KriMonitor {
    kri_repo: Arc<dyn KriRepository>,
    kernel:   Arc<dyn KernelFacade>,
}

impl KriMonitor {
```

```
    /// Called periodically from the kernel scheduler.
    pub async fn check_all_kris(&self) -> Result<(), UmeError> {
        let kris = self.kri_repo.all_active().await?;

        for kri in kris {
            let Some(value) = kri.current_value else { continue };

            let status = if value >= kri.red_threshold {
                KriStatus::Red
            } else if value >= kri.amber_threshold {
                KriStatus::Amber
            } else {
                KriStatus::Green
            };

            match status {
                KriStatus::Red => {
                    self.kernel.publish("risk.kri.red_threshold", json!({
                        "kri_id": kri.id, "risk_id": kri.risk_id,
                        "value": value, "threshold": kri.red_threshold,
                    }), Default::default()).await?;
                }
                KriStatus::Amber => {
                    self.kernel.publish("risk.kri.amber_threshold", json!({
                        "kri_id": kri.id, "risk_id": kri.risk_id,
                        "value": value, "threshold": kri.amber_threshold,
                    }), Default::default()).await?;
                }
                KriStatus::Green => {}
            }
        }
        Ok(())
    }
}
```

6.5 Process, Orchestration & Workflow Module (MOD-29)

6.5.1 Workflow State Machine

Workflow instances are modeled as state machines. The Process Module persists instance state and transitions; the executor pool handles step execution.

SQL / DDL

```
-- process.workflow_definitions
CREATE TABLE process.workflow_definitions (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id       UUID NOT NULL,
    name         VARCHAR(200) NOT NULL,
    version      INT NOT NULL DEFAULT 1,
    trigger_event VARCHAR(100),  -- event topic that auto-starts this workflow
    steps        JSONB NOT NULL, -- serialized WorkflowStep[]
    is_active    BOOLEAN NOT NULL DEFAULT true
);

-- process.workflow_instances
CREATE TABLE process.workflow_instances (
    id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id         UUID NOT NULL,
```

```
    definition_id  UUID NOT NULL REFERENCES process.workflow_definitions(id),
    status         VARCHAR(20) NOT NULL DEFAULT 'Running'
                   CHECK (status IN ('Running','Suspended','Completed',
                                     'Failed','Cancelled')),
    context        JSONB NOT NULL DEFAULT '{}'::jsonb,  -- runtime variables
    current_step   VARCHAR(100),
    started_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at   TIMESTAMPTZ,
    triggered_by   UUID,     -- source event ID
    parent_instance_id UUID REFERENCES process.workflow_instances(id)
);
```

6.5.2 Workflow Executor

Rust

```rust
// ume_process/src/executor.rs

pub struct WorkflowExecutor {
    instance_repo:    Arc<dyn InstanceRepository>,
    definition_repo:  Arc<dyn DefinitionRepository>,
    task_dispatcher:  Arc<TaskDispatcher>,
    kernel:           Arc<dyn KernelFacade>,
}

impl WorkflowExecutor {
    pub async fn advance(&self, instance_id: &InstanceId) -> Result<(), UmeError> {
        let mut instance = self.instance_repo.get(instance_id).await?
            .ok_or_else(|| UmeError::not_found("WorkflowInstance", instance_id))?;
        let definition = self.definition_repo.get(&instance.definition_id).await?
            .ok_or_else(|| UmeError::not_found("WorkflowDefinition",
&instance.definition_id))?;

        let current_step = definition.step_by_id(&instance.current_step)?;

        match &current_step.step_type {
            StepType::HumanTask { assignee_expr, form_id, sla_hours } => {
                // Create work item in Work Module via event
                self.kernel.publish("process.task.created", json!({
                    "instance_id": instance_id,
                    "step_id": current_step.id,
                    "assignee": evaluate_expr(assignee_expr, &instance.context),
                    "form_id": form_id,
                    "due_by": Utc::now() + Duration::hours(*sla_hours as i64),
                }), Default::default()).await?;
            }
            StepType::AutomatedTask { action } => {
                let result = self.execute_action(action, &instance.context).await?;
                instance.context.merge(result);
                self.transition_to_next(&mut instance, &definition, None).await?;
            }
            StepType::Decision { conditions } => {
                let next = conditions.iter()
                    .find(|c| c.condition.evaluate(&instance.context))
                    .map(|c| &c.next_step_id)
                    .ok_or_else(|| UmeError::business_rule("no matching decision
branch"))?;
                self.transition_to_named_step(&mut instance, &definition,
next).await?;
            }
            StepType::End => {
```

```
                self.complete_instance(&mut instance).await?;
            }
        }
        Ok(())
    }
}
```

6.6 Marketing System — Soko (MOD-22)

6.6.1 Strategy Pack Engine

Soko's core capability is executing 70+ strategy packs against a campaign and its accumulated market signals. Each pack is a rule-based evaluator that returns insights and activation actions.

SQL / DDL

```
-- soko.campaigns
CREATE TABLE soko.campaigns (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id          UUID NOT NULL,
    name            VARCHAR(200) NOT NULL,
    status          VARCHAR(20) NOT NULL DEFAULT 'Draft'
                    CHECK (status IN ('Draft','Active','Paused','Completed')),
    channels        TEXT[] NOT NULL,
    target_audience JSONB,      -- audience persona definition
    budget_amount   NUMERIC(12,2),
    budget_currency CHAR(3) DEFAULT 'USD',
    start_date      DATE,
    end_date        DATE,
    kpi_targets     JSONB,      -- { cac: 50, conversion_rate: 0.05, ... }
    strategy_context JSONB,    -- market context fed to strategy packs
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- soko.market_signals
CREATE TABLE soko.market_signals (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id          UUID NOT NULL,
    campaign_id     UUID REFERENCES soko.campaigns(id),
    signal_type     VARCHAR(50) NOT NULL, -- Click|Open|Conversion|Impression|...
    channel         VARCHAR(50),
    value           NUMERIC(12,4),
    metadata        JSONB,
    received_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Rust

```
// ume_soko/src/services/strategy_engine.rs

pub struct StrategyEngine {
    packs:          Vec<Arc<dyn StrategyPack>>,  // 70+ strategy packs loaded at
init
    signal_repo:    Arc<dyn SignalRepository>,
    kernel:         Arc<dyn KernelFacade>,
}

/// Each strategy pack implements this trait.
```

```rust
pub trait StrategyPack: Send + Sync {
    fn pack_id(&self) -> &'static str;
    fn name(&self) -> &'static str;
    fn category(&self) -> StrategyCategory;

    /// Evaluate the pack against the campaign + accumulated signals.
    /// Returns insights (human-readable) and actions (machine-executable).
    fn evaluate(&self, campaign: &Campaign,
                signals: &[MarketSignal]) -> PackEvalResult;
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct PackEvalResult {
    pub pack_id:    String,
    pub insights:   Vec<Insight>,     // human-readable findings
    pub actions:    Vec<ActivationAction>,  // recommended next actions
    pub score:      f32,              // 0.0-1.0 relevance score for this campaign
}

impl StrategyEngine {
    /// Evaluate all packs in parallel. Returns ranked insights + action list.
    pub async fn evaluate_campaign(&self, campaign: &Campaign)
            -> Result<StrategyEvaluation, UmeError> {
        let signals = self.signal_repo
            .list_by_campaign(&campaign.id).await?;

        // CPU-bound: run all 70+ packs in parallel via rayon
        let results: Vec<PackEvalResult> = self.packs.par_iter()
            .map(|pack| pack.evaluate(campaign, &signals))
            .collect();

        // Rank by score, deduplicate overlapping insights
        let ranked = Self::rank_and_deduplicate(results);

        self.kernel.publish("soko.campaign.evaluated", json!({
            "campaign_id": campaign.id,
            "insight_count": ranked.insights.len(),
            "top_action": ranked.actions.first(),
        }), Default::default()).await?;

        Ok(ranked)
    }
}
```

6.7 Security, Privacy & Audit Module (MOD-36)

6.7.1 Identity & Session Management

SQL / DDL

```sql
-- security.identities
CREATE TABLE security.identities (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id          UUID NOT NULL,
    employee_id     UUID REFERENCES hr.employees(id),
    username        VARCHAR(254) NOT NULL,
    email           VARCHAR(254) NOT NULL,
    password_hash   VARCHAR(200),          -- bcrypt($cost=12) or NULL if SSO-only
    mfa_enabled     BOOLEAN NOT NULL DEFAULT false,
```

```
    mfa_type        VARCHAR(20),           -- TOTP | FIDO2 | SMS
    mfa_secret      BYTEA,                 -- encrypted; vault-managed key
    status          VARCHAR(20) NOT NULL DEFAULT 'Active'
                    CHECK (status IN ('Active','Locked','Disabled')),
    failed_attempts SMALLINT NOT NULL DEFAULT 0,
    locked_until    TIMESTAMPTZ,
    last_login_at   TIMESTAMPTZ,
    password_changed_at TIMESTAMPTZ
);

-- security.sessions (JWT nonce store for revocation)
CREATE TABLE security.sessions (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    identity_id     UUID NOT NULL REFERENCES security.identities(id),
    jti             UUID NOT NULL UNIQUE,   -- JWT ID claim
    issued_at       TIMESTAMPTZ NOT NULL,
    expires_at      TIMESTAMPTZ NOT NULL,
    revoked_at      TIMESTAMPTZ,
    ip_address      INET,
    user_agent      VARCHAR(500)
);
```

6.7.2 Authentication Flow

Rust

```rust
// ume_security/src/services/auth_service.rs

impl AuthService {
    pub async fn authenticate(&self, req: AuthRequest)
            -> Result<TokenPair, UmeError> {
        // 1. Load identity
        let identity = self.identity_repo
            .by_username(&req.username).await?
            .ok_or(UmeError::Unauthenticated)?;

        // 2. Lockout check
        if let Some(locked_until) = identity.locked_until {
            if Utc::now() < locked_until {
                return Err(UmeError::BusinessRuleViolation {
                    rule: "account.lockout".into(),
                    message: format!("Account locked until {}", locked_until),
                });
            }
        }

        // 3. Password verification (constant-time)
        let valid = bcrypt::verify(&req.password, &identity.password_hash
            .ok_or(UmeError::Unauthenticated)?)?;

        if !valid {
            self.record_failed_attempt(&identity.id).await?;
            return Err(UmeError::Unauthenticated);
        }

        // 4. MFA challenge (if enrolled)
        if identity.mfa_enabled {
            if let Some(totp) = req.totp_code {
                self.verify_totp(&identity, &totp)?;
            } else {
                return Err(UmeError::MfaRequired);
```

```
            }
        }

        // 5. Reset failed attempts; issue tokens
        self.reset_failed_attempts(&identity.id).await?;
        let tokens = self.issue_tokens(&identity).await?;

        // 6. Audit
        self.kernel.audit(AuditRecord::new(
            &SubjectId::from(&identity.id),
            "security.identity.login",
            &identity.id.to_string(),
            AuditOutcome::Success,
        )).await?;

        Ok(tokens)
    }
}
```

6.8 Work Management Module (MOD-40)

6.8.1 Work Item Model

SQL / DDL

```
-- work.items
CREATE TABLE work.items (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id          UUID NOT NULL,
    item_ref        VARCHAR(30),             -- WI-2026-001
    title           VARCHAR(500) NOT NULL,
    description     TEXT,
    item_type       VARCHAR(30) NOT NULL
                    CHECK (item_type IN ('Task','Story','Bug','Epic',
                                         'WorkOrder','Approval','Request')),
    status          VARCHAR(30) NOT NULL DEFAULT 'Backlog'
                    CHECK (status IN ('Backlog','ToDo','InProgress',
                                      'InReview','Done','Cancelled')),
    priority        SMALLINT NOT NULL DEFAULT 3 CHECK (priority BETWEEN 1 AND 5),
    assignee_id     UUID REFERENCES hr.employees(id),
    org_unit_id     UUID REFERENCES admin.org_units(id),
    project_id      UUID REFERENCES portfolio.projects(id),
    sprint_id       UUID REFERENCES work.sprints(id),
    parent_id       UUID REFERENCES work.items(id),
    story_points    SMALLINT,
    due_date        DATE,
    started_at      TIMESTAMPTZ,
    completed_at    TIMESTAMPTZ,
    estimated_hours NUMERIC(6,2),
    logged_hours    NUMERIC(6,2) GENERATED ALWAYS AS (
        (SELECT COALESCE(SUM(hours),0) FROM work.time_logs WHERE item_id = id)
    ) STORED,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    version         BIGINT NOT NULL DEFAULT 1
);
```

7. API Layer Design

7.1 ume_server Architecture

ume_server is an Axum-based HTTP server. It receives requests, authenticates, authorizes, routes to module handlers, and returns structured responses. It does not contain business logic — all logic lives in kernel modules.

Rust

```rust
// ume_server/src/main.rs

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let config = KernelConfig::from_env()?;
    let kernel = UmeKernel::boot(config).await?;

    let app = Router::new()
        // System routes
        .route("/health",       get(handlers::system::health))
        .route("/metrics",      get(handlers::system::metrics))
        .route("/openapi.json", get(handlers::system::openapi))

        // Auth routes (no JWT required)
        .route("/v1/auth/login",    post(handlers::auth::login))
        .route("/v1/auth/refresh",  post(handlers::auth::refresh))
        .route("/v1/auth/logout",   post(handlers::auth::logout))

        // Module API routes (JWT required via middleware)
        .nest("/v1/finance",     finance_router())
        .nest("/v1/hr",          hr_router())
        .nest("/v1/chombo",      chombo_router())
        .nest("/v1/grc",         grc_router())
        .nest("/v1/risk",        risk_router())
        .nest("/v1/sales",       sales_router())
        .nest("/v1/work",        work_router())
        // ... 35 more module routers

        // Middleware stack
        .layer(middleware::from_fn_with_state(kernel.clone(), auth_middleware))
        .layer(middleware::from_fn(request_id_middleware))
        .layer(middleware::from_fn(rate_limit_middleware))
        .layer(middleware::from_fn(logging_middleware))
        .layer(TraceLayer::new_for_http())
        .with_state(kernel.clone());

    let addr = SocketAddr::from(([0, 0, 0, 0], config.server.port));
    tracing::info!("ume_server listening on {}", addr);
    axum::Server::bind(&addr)
        .serve(app.into_make_service_with_connect_info::<SocketAddr>())
        .with_graceful_shutdown(shutdown_signal())
        .await?;
    Ok(())
}
```

7.2 Authentication Middleware

Rust

```rust
// ume_server/src/middleware/auth.rs

pub async fn auth_middleware(
    State(kernel): State<Arc<UmeKernel>>,
    mut request: Request,
    next: Next,
) -> Result<Response, ApiError> {
    // Skip auth for public routes
    if is_public_path(request.uri().path()) {
        return Ok(next.run(request).await);
    }

    // Extract bearer token
    let token = request.headers()
        .get(AUTHORIZATION)
        .and_then(|v| v.to_str().ok())
        .and_then(|v| v.strip_prefix("Bearer "))
        .ok_or(ApiError::Unauthenticated)?;

    // Verify JWT (RS256; public key from Security Module)
    let claims = kernel.security_module()
        .verify_token(token).await
        .map_err(|_| ApiError::Unauthenticated)?;

    // Check token revocation (session store)
    if kernel.security_module().is_revoked(&claims.jti).await? {
        return Err(ApiError::Unauthenticated);
    }

    // Inject authenticated context into request extensions
    request.extensions_mut().insert(AuthContext {
        subject_id: claims.sub.parse()?,
        org_id:     claims.org_id.parse()?,
        session_id: claims.jti.to_string(),
        roles:      claims.roles,
    });

    Ok(next.run(request).await)
}
```

7.3 Standard Handler Pattern

All module handlers follow an identical pattern: extract auth context → extract and validate request → call service → map result to response → return.

Rust

```rust
// ume_server/src/handlers/finance.rs

pub async fn post_journal_entry(
    State(kernel): State<Arc<UmeKernel>>,
    Extension(auth): Extension<AuthContext>,
    Json(body): Json<PostJournalRequest>,
) -> Result<Json<PostJournalResponse>, ApiError> {

    // 1. Input validation (serde + custom validators)
    body.validate().map_err(ApiError::validation)?;
```

```
    // 2. Route to module service
    let finance = kernel.module::<FinanceModule>("finance.ledger")?;
    let entry_id = finance.ledger_service()
        .post_journal(body.into_command(), &auth.subject_id)
        .await
        .map_err(ApiError::from)?;

    // 3. Return structured response
    Ok(Json(PostJournalResponse {
        id: entry_id,
        status: "posted",
    }))
}

// ApiError maps UmeError variants to HTTP status codes
impl From<UmeError> for ApiError {
    fn from(err: UmeError) -> Self {
        match err {
            UmeError::AuthorizationFailed { .. } =>
ApiError::Forbidden(err.to_string()),
            UmeError::Unauthenticated         => ApiError::Unauthenticated,
            UmeError::NotFound { .. }          =>
ApiError::NotFound(err.to_string()),
            UmeError::ValidationFailed { .. }  =>
ApiError::UnprocessableEntity(err),
            UmeError::OptimisticLockConflict{..}=>
ApiError::Conflict(err.to_string()),
            UmeError::BusinessRuleViolation{..} =>
ApiError::UnprocessableEntity(err),
            _                                  =>
ApiError::Internal(err.to_string()),
        }
    }
}
```

7.4 Standard API Response Envelopes

All API responses follow consistent envelope structures for both success and error cases.

Rust

```
// ume_server/src/response.rs — response envelope types

/// Single resource response
#[derive(Serialize)]
pub struct ResourceResponse<T: Serialize> {
    pub data:    T,
    pub meta:    ResponseMeta,
}

/// Paginated list response
#[derive(Serialize)]
pub struct ListResponse<T: Serialize> {
    pub data:       Vec<T>,
    pub pagination: PaginationMeta,
    pub meta:       ResponseMeta,
}
```

```
#[derive(Serialize)]
pub struct PaginationMeta {
    pub cursor:      Option<String>,  // next page cursor
    pub has_more:    bool,
    pub total_count: Option<u64>,     // only included when explicitly requested
}

#[derive(Serialize)]
pub struct ResponseMeta {
    pub request_id:   String,
    pub api_version:  &'static str,
    pub generated_at: DateTime<Utc>,
}

/// Error response
#[derive(Serialize)]
pub struct ErrorResponse {
    pub error_code:       String,           // machine-readable e.g.
VALIDATION_FAILED
    pub message:          String,           // human-readable summary
    pub detail:           Vec<FieldError>,  // field-level errors
    pub request_id:       String,
    pub documentation_url: Option<String>,
}
```

7.5 WebSocket Event Stream

Real-time dashboard updates are delivered via WebSocket. Clients authenticate with a JWT query param and subscribe to event topic patterns.

Rust

```
// ume_server/src/ws/event_stream.rs

pub async fn ws_handler(
    ws: WebSocketUpgrade,
    State(kernel): State<Arc<UmeKernel>>,
    Query(params): Query<WsParams>,
) -> Response {
    // Authenticate via token query param (WS can't use headers)
    let auth = kernel.security_module()
        .verify_token(&params.token).await
        .expect("ws auth failed");

    ws.on_upgrade(move |socket| handle_ws(socket, kernel, auth))
}

async fn handle_ws(mut socket: WebSocket, kernel: Arc<UmeKernel>,
                   auth: TokenClaims) {
    // Client sends subscribe messages: { "action": "subscribe", "topic":
"risk.kri.*" }
    let mut subscriptions: Vec<SubscriptionHandle> = Vec::new();
    let (tx, mut rx) = tokio::sync::mpsc::channel::<Message>(100);

    loop {
        tokio::select! {
            Some(msg) = socket.recv() => {
                match parse_client_message(msg) {
                    ClientMessage::Subscribe { topic } => {
```

```
                        let handle = kernel.event_bus()
                            .subscribe(&topic, Arc::new(WsForwarder(tx.clone())))
                            .await.unwrap();
                        subscriptions.push(handle);
                    }
                    ClientMessage::Unsubscribe { topic } => {
                        subscriptions.retain(|h| h.topic() != &topic);
                    }
                    ClientMessage::Ping => {
                        socket.send(Message::Pong(vec![])).await.ok();
                    }
                    ClientMessage::Close => break,
                }
            }
            Some(event_msg) = rx.recv() => {
                if socket.send(event_msg).await.is_err() { break; }
            }
        }
    }
    // Clean up subscriptions on disconnect
    for handle in subscriptions { handle.cancel().await; }
}
```

7.6 ume_runtime CLI Design

The CLI runtime is a Tokio-based REPL and batch runner. It connects to the kernel in-process (not via HTTP) for system operations.

Rust

```rust
// ume_runtime/src/main.rs

fn main() -> anyhow::Result<()> {
    let cli = Cli::parse();  // clap

    match cli.command {
        Commands::Start    => runtime::start_server()?,
        Commands::Repl     => runtime::start_repl()?,
        Commands::Exec { script } => runtime::exec_script(script)?,
        Commands::Kernel(cmd) => runtime::kernel_command(cmd)?,
        Commands::Module(cmd) => runtime::module_command(cmd)?,
        Commands::Backup(cmd) => runtime::backup_command(cmd)?,
        Commands::Audit(cmd)  => runtime::audit_command(cmd)?,
    }
    Ok(())
}

// Example REPL interaction
// ume> kernel status
// State: Running | Boot time: 2026-03-07T08:00:00Z (4h 12m ago)
// Modules: 40 running, 0 degraded, 0 stopped
// Drivers: storage=healthy identity=healthy comms=healthy
//
// ume> module list --domain=finance
// ID                  Domain   Version  State    Last Health
// finance.ledger      Finance  1.0.0    Running  OK (28s ago)
//
// ume> audit query --actor=user:alice --from=2026-03-01 --limit=20
```

8. Data Architecture

8.1 Schema Namespace Organization

Each module owns its own PostgreSQL schema. Cross-module foreign keys reference other schemas explicitly. This enforces clear data ownership at the database level.

Schema	Owner Module	Key Tables
admin	MOD-01 Administration	organizations, org_units, policies, enterprise_calendar
finance	MOD-14 Finance	ledger_accounts, journal_entries, journal_lines, invoices, budget_lines, periods
hr	MOD-16 HR	employees, compensation, leave_balances, leave_requests, payroll_runs
chombo	MOD-13 Legal Entity	legal_entities, policy_packs, filing_records, compliance_alerts
grc	MOD-15 GRC	obligations, controls, control_tests, audits, audit_findings
risk	MOD-33 Risk	risks, treatment_plans, kris, kri_readings
security	MOD-36 Security	identities, sessions, incidents, vulnerabilities, privacy_records
work	MOD-40 Work	items, sprints, time_logs, dependencies
process	MOD-29 Process	workflow_definitions, workflow_instances, workflow_steps
crm	MOD-08 CRM	accounts, contacts, interactions, relationships
sales	MOD-34 Sales	opportunities, pipeline_stages, quotes, commissions
soko	MOD-22 Marketing	campaigns, market_signals, strategy_pack_results, channels
portfolio	MOD-27 Portfolio	portfolios, programs, projects, milestones
it	MOD-18 IT	configuration_items, incidents, changes, service_requests
cms	MOD-06 CMS	documents, document_versions, document_access
learning	MOD-20 Learning	courses, learning_paths, enrollments, completions, certificates
investment	MOD-17 Investment	funds, positions, appraisals, capital_accounts
audit	Kernel	audit_records (append-only, no delete path)
migrations	Kernel	migration_log (applied migration tracking)

8.2 Multi-Tenancy Design

Multi-tenancy is enforced via org_id partitioning. Every data table carries org_id as a NOT NULL column. All queries are org_id-scoped in the StorageManager layer.

Rust

```rust
// ume_kernel/src/storage.rs — multi-tenancy enforcement

impl TieredStorageManager {
    /// All queries are automatically scoped to the current org.
    pub async fn query<T: DeserializeOwned + Send>(
        &self,
        namespace: &str,
        filter: Filter,
        page: PageRequest,
```

```
    ) -> Result<PageResponse<T>, UmeError> {
        // Inject org_id into every query — this is unconditional
        let scoped_filter = filter.and(Filter::eq("org_id",
self.org_id.to_string()));

        self.warm_store.query(namespace, scoped_filter, page).await
    }

    pub async fn put<T: Serialize + Send>(
        &self,
        namespace: &str,
        id: &str,
        value: &T,
        version: Option<u64>,
    ) -> Result<u64, UmeError> {
        // All values must contain org_id field — enforced at schema level
        self.warm_store.put(namespace, id, value, version).await
    }
}
```

8.3 Core Schema Conventions

All domain entity tables follow these conventions without exception:

Convention	Column Name	Type	Description
Primary key	id	UUID DEFAULT gen_random_uuid()	UUID v4 generated by the database
Tenant scope	org_id	UUID NOT NULL	Owning organization; all queries scoped by this
Optimistic lock	version	BIGINT NOT NULL DEFAULT 1	Incremented on every UPDATE; stale writes rejected
Created timestamp	created_at	TIMESTAMPTZ NOT NULL DEFAULT NOW()	UTC timestamp; never modified after insert
Updated timestamp	updated_at	TIMESTAMPTZ NOT NULL DEFAULT NOW()	UTC timestamp; updated by trigger on every UPDATE
Created by	created_by	UUID	SubjectId of the actor who created the record
Soft delete	deleted_at	TIMESTAMPTZ	NULL = active; non-NULL = soft-deleted
Soft delete actor	deleted_by	UUID	SubjectId of the actor who deleted the record

8.4 Migration System

All schema changes are applied through the StorageManager's migration system. Migrations are versioned, idempotent, and recorded in the migration_log table.

SQL / DDL

```
-- migrations.migration_log (managed by kernel; never touched by modules)
CREATE TABLE migrations.migration_log (
    id              SERIAL PRIMARY KEY,
```

```
    migration_id   VARCHAR(100) UNIQUE NOT NULL,  -- e.g.
"finance_0012_add_fund_type"
    module_id      VARCHAR(100) NOT NULL,
    applied_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    checksum       VARCHAR(64) NOT NULL,          -- SHA-256 of migration SQL
    duration_ms    INT NOT NULL
);
```

Rust

```
// Migration file naming: {module}_{seq:04}_{description}.sql
// e.g. finance_0001_initial_schema.sql
//      finance_0002_add_restricted_funds.sql
//      chombo_0001_initial_schema.sql

// Migrations are registered in each module's lib.rs:
pub const MIGRATIONS: &[Migration] = &[
    Migration {
        id:       "finance_0001_initial_schema",
        module:   "finance.ledger",
        up:       include_str!("../migrations/finance_0001_initial_schema.sql"),
        checksum: "a3f7c2...",  // compile-time SHA-256
    },
    Migration {
        id:       "finance_0002_add_restricted_funds",
        module:   "finance.ledger",
        up:
include_str!("../migrations/finance_0002_add_restricted_funds.sql"),
        checksum: "b8e1d4...",
    },
];
```

8.5 Event Persistence & Replay

High-value domain events are persisted to the event_log table in addition to being dispatched on the bus. This enables audit replay and event sourcing for Finance, GRC, and Board modules.

SQL / DDL

```
-- kernel.event_log (persisted events for audit and replay)
CREATE TABLE kernel.event_log (
    id               UUID PRIMARY KEY,
    topic            VARCHAR(200) NOT NULL,
    payload          JSONB NOT NULL,
    publisher_module VARCHAR(100) NOT NULL,
    published_at     TIMESTAMPTZ NOT NULL,
    correlation_id   VARCHAR(100),
    causation_id     UUID REFERENCES kernel.event_log(id),
    org_id           UUID NOT NULL
);

CREATE INDEX idx_event_log_topic ON kernel.event_log(topic);
CREATE INDEX idx_event_log_published_at ON kernel.event_log(published_at);
CREATE INDEX idx_event_log_org_id ON kernel.event_log(org_id);
```

8.6 Full-Text Search Architecture

The Knowledge and CMS modules require full-text search. Search is implemented via a dedicated SearchDriver that abstracts the backing search engine (PostgreSQL tsvector, or Elasticsearch/Meilisearch for scale).

Rust

```rust
// ume_core/src/traits.rs — SearchDriver trait

#[async_trait]
pub trait SearchDriver: Send + Sync {
    /// Index a document for searching.
    async fn index(&self, req: IndexRequest) -> Result<(), UmeError>;

    /// Delete a document from the index.
    async fn delete(&self, namespace: &str, id: &str) -> Result<(), UmeError>;

    /// Full-text search with optional filter and ranking.
    async fn search<T: DeserializeOwned + Send>(
        &self,
        query: SearchQuery,
    ) -> Result<SearchResponse<T>, UmeError>;

    /// Suggest completions for typeahead.
    async fn suggest(&self, prefix: &str, namespace: &str)
        -> Result<Vec<String>, UmeError>;
}

#[derive(Debug, Serialize, Deserialize)]
pub struct IndexRequest {
    pub namespace:  String,       // e.g. "knowledge.articles"
    pub id:         String,
    pub title:      String,
    pub body:       String,
    pub tags:       Vec<String>,
    pub metadata:   serde_json::Value,
    pub org_id:     OrgId,
}
```

9. Security Architecture Design

9.1 Security Layering Model

UME implements defence-in-depth with four concentric security layers. An attacker must defeat every layer to reach sensitive data.

Layer	Mechanism	Where Enforced	Bypassed By
Transport	TLS 1.2+ on all HTTP connections	ume_server TLS termination	Nothing — TLS is mandatory in production mode
Authentication	JWT RS256 + MFA	auth_middleware; SecurityModule.verify_token()	Nothing — all non-public routes require valid JWT
Authorization	RBAC permission check	KernelFacade.assert_can(); called in every service method	Nothing — kernel facade enforces before reaching module logic
Data	AES-256-GCM at rest; field-level encryption for PII	StorageDriver encryption; PII field decorators	Physical database access still sees ciphertext only

9.2 JWT Token Design

Access tokens are RS256-signed JWTs. The signing key pair is generated and stored in the Security Module vault. Public keys are distributed to ume_server instances via the key distribution endpoint.

Rust

```rust
// JWT Claims structure
#[derive(Debug, Serialize, Deserialize)]
pub struct UmeClaims {
    // Standard JWT claims
    pub iss: String,          // "ume:{org_id}"
    pub sub: String,          // SubjectId (UUID)
    pub aud: String,          // "ume-api"
    pub iat: i64,             // issued at (Unix timestamp)
    pub exp: i64,             // expiry (Unix timestamp)
    pub jti: Uuid,            // unique token ID for revocation

    // UME-specific claims
    pub org_id: String,       // OrgId
    pub subject_type: String, // User | ServiceAccount | ApiKey
    pub roles: Vec<String>,   // role IDs (for fast RBAC pre-check)
    pub mfa_verified: bool,   // whether MFA was completed this session
}

// Token issuance (Security Module)
impl AuthService {
    async fn issue_tokens(&self, identity: &Identity) -> Result<TokenPair,
UmeError> {
        let now = Utc::now();
        let access_claims = UmeClaims {
            iss: format!("ume:{}", self.org_id),
            sub: identity.id.to_string(),
            aud: "ume-api".into(),
            iat: now.timestamp(),
            exp: (now + Duration::minutes(60)).timestamp(),
            jti: Uuid::new_v4(),
```

```
            org_id: self.org_id.to_string(),
            subject_type: "User".into(),
            roles: self.rbac.roles_for(&identity.id).await?,
            mfa_verified: identity.mfa_enabled,
        };

        let access_token = jsonwebtoken::encode(
            &Header::new(Algorithm::RS256),
            &access_claims,
            &self.vault.signing_key().await?,
        )?;

        // Persist session for revocation support
        self.session_repo.create(&Session {
            identity_id: identity.id,
            jti: access_claims.jti,
            issued_at: now,
            expires_at: now + Duration::minutes(60),
            ..Default::default()
        }).await?;

        Ok(TokenPair { access_token, refresh_token: /* ... */ })
    }
}
```

9.3 Cryptographic Vault Interface

No module ever handles raw key material. All cryptographic operations go through the vault interface provided by the Security Module. This design is compatible with HSM backends.

Rust

```rust
// ume_core/src/traits.rs — CryptoVault trait

#[async_trait]
pub trait CryptoVault: Send + Sync {
    /// Sign data with the named key.
    async fn sign(&self, key_id: &str, data: &[u8]) -> Result<Vec<u8>, UmeError>;

    /// Verify a signature.
    async fn verify(&self, key_id: &str, data: &[u8],
                    sig: &[u8]) -> Result<bool, UmeError>;

    /// Encrypt data with the named key (AES-256-GCM).
    async fn encrypt(&self, key_id: &str, plaintext: &[u8])
        -> Result<EncryptedBlob, UmeError>;

    /// Decrypt data with the named key.
    async fn decrypt(&self, key_id: &str, blob: &EncryptedBlob)
        -> Result<Vec<u8>, UmeError>;

    /// Rotate a key. Returns the new key ID.
    /// Old key remains available for decryption during rotation window.
    async fn rotate_key(&self, key_id: &str) -> Result<String, UmeError>;

    /// Returns the public portion of an asymmetric key (for JWT verification).
    async fn public_key(&self, key_id: &str) -> Result<Vec<u8>, UmeError>;
}
```

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct EncryptedBlob {
    pub key_id:     String,    // identifies which key was used
    pub key_version: u32,      // supports key rotation
    pub nonce:      Vec<u8>,   // 96-bit random nonce (GCM)
    pub ciphertext: Vec<u8>,
    pub tag:        Vec<u8>,   // 128-bit GCM authentication tag
}
```

9.4 Input Validation Architecture

All API inputs pass through three validation tiers before reaching business logic.

Tier	Mechanism	What It Catches	Returns On Failure
Schema validation	serde + validator crate derive macros	Type mismatches; missing required fields; basic format constraints	HTTP 422 with field-level FieldError[]
Domain validation	Service-layer validate() method calls	Business constraint violations; cross-field rules; referential integrity pre-checks	UmeError::ValidationFailed → HTTP 422
Persistence validation	Database constraints (CHECK, FK, UNIQUE)	Race conditions; concurrent violations; deeper invariants	UmeError::StorageError → HTTP 409 or 500

Rust

```rust
// Tier 1: serde + validator derive macros
#[derive(Debug, Deserialize, Validate)]
pub struct PostJournalRequest {
    #[validate(length(min = 1, max = 200))]
    pub description: String,

    #[validate(length(min = 2, message = "At least 2 lines required"))]
    pub lines: Vec<JournalLineRequest>,

    #[validate(regex(path = "PERIOD_REGEX", message = "Must be YYYY-MM"))]
    pub period: String,
}

// Tier 2: domain validation in service layer
impl PostJournalRequest {
    pub fn validate_domain(&self) -> Result<(), UmeError> {
        let sum: Decimal = self.lines.iter()
            .map(|l| l.debit.unwrap_or_default() - l.credit.unwrap_or_default())
            .sum();
        if sum != Decimal::ZERO {
            return Err(UmeError::ValidationFailed {
                errors: vec![FieldError {
                    field: "lines".into(),
                    message: format!("Journal does not balance: net = {}", sum),
                }]
            });
        }
        Ok(())
    }
}
```

9.5 SQL Injection Prevention

All database interactions use parameterized queries. The StorageManager enforces this at the trait level — there is no API surface for raw SQL string concatenation from module code.

Rust

```rust
// Filter types force parameterization — no raw SQL from modules

#[derive(Debug, Clone)]
pub enum Filter {
    Eq(String, FilterValue),
    Ne(String, FilterValue),
    Gt(String, FilterValue),
    Lt(String, FilterValue),
    In(String, Vec<FilterValue>),
    Like(String, String),      // pattern is never user-controlled
    IsNull(String),
    And(Vec<Filter>),
    Or(Vec<Filter>),
}

#[derive(Debug, Clone)]
pub enum FilterValue {
    Str(String),
    Uuid(Uuid),
    Int(i64),
    Decimal(Decimal),
    Bool(bool),
    DateTime(DateTime<Utc>),
}

// The SQLite/Postgres backend converts Filter to parameterized SQL internally.
// Modules never see or write SQL — they use Filter enum values only.
```

10. Device Driver Design

10.1 Driver Trait & Lifecycle

All external system integrations implement the DeviceDriver trait. The kernel manages driver lifecycle. Modules never hold references to drivers directly — they access them through the kernel facade.

Rust

```rust
// ume_core/src/traits.rs — DeviceDriver base trait

#[async_trait]
pub trait DeviceDriver: Send + Sync + std::fmt::Debug {
    fn device_id(&self)    -> &'static str;
    fn device_class(&self) -> DeviceClass;
    fn status(&self)       -> DriverStatus;

    async fn initialize(&mut self, config: &DriverConfig) -> Result<(), UmeError>;
    async fn shutdown(&mut self)                          -> Result<(), UmeError>;
    async fn health_check(&self)                         -> DriverHealth;
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum DeviceClass {
    Storage,   // StorageDriver
    Identity,  // IdentityDriver (LDAP, OAuth2, SAML)
    Comms,     // CommDriver (SMTP, SMS, push)
    Broker,    // EventBrokerDriver (NATS, Kafka, RabbitMQ)
    Finance,   // FinanceDriver (ERP, banking)
    Search,    // SearchDriver (Elasticsearch, Meilisearch)
    Crypto,    // CryptoDriver (vault, HSM)
    Storage2,  // ObjectStorage (S3, GCS, Azure Blob)
}
```

10.2 Storage Driver

Rust

```rust
// ume_kernel/src/drivers/storage/

// SQLite driver — for embedded single-process deployment
pub struct SqliteStorageDriver {
    pool:    sqlx::SqlitePool,
    db_path: PathBuf,
}

// PostgreSQL driver — for standard and enterprise deployments
pub struct PostgresStorageDriver {
    pool:       sqlx::PgPool,
    read_pools: Vec<sqlx::PgPool>,  // read replicas
    config:     PostgresConfig,
}

impl PostgresStorageDriver {
    /// Route read vs write to appropriate pool.
    fn pool_for(&self, is_write: bool) -> &sqlx::PgPool {
        if is_write || self.read_pools.is_empty() {
```

```
            &self.pool
        } else {
            // Round-robin over read replicas
            let idx = self.read_counter.fetch_add(1, Ordering::Relaxed)
                % self.read_pools.len();
            &self.read_pools[idx]
        }
    }
}

// InMemory driver — for tests and ephemeral dev environments
pub struct InMemoryStorageDriver {
    store: Arc<DashMap<String, (serde_json::Value, u64)>>,  // key → (value,
version)
}
```

10.3 Identity Driver

Rust

```
// Supports multiple identity backend protocols

#[async_trait]
pub trait IdentityDriver: DeviceDriver {
    /// Authenticate credentials against the identity backend.
    async fn authenticate(&self, username: &str, password: &str)
        -> Result<Option<ExternalIdentity>, UmeError>;

    /// Resolve an OAuth2/SAML token to an external identity.
    async fn resolve_token(&self, token: &str)
        -> Result<Option<ExternalIdentity>, UmeError>;

    /// Sync group memberships for a user.
    async fn groups_for(&self, external_id: &str)
        -> Result<Vec<String>, UmeError>;
}

// Available implementations:
// - LocalIdentityDriver    — bcrypt password store in UME DB (default)
// - LdapIdentityDriver     — LDAP/Active Directory integration
// - OidcIdentityDriver     — OpenID Connect (Google, Azure AD, Okta)
// - SamlIdentityDriver     — SAML 2.0 (enterprise SSO)
```

10.4 Communications Driver

Rust

```
#[async_trait]
pub trait CommDriver: DeviceDriver {
    async fn send_email(&self, msg: EmailMessage) -> Result<String, UmeError>;
    async fn send_sms(&self,   msg: SmsMessage)   -> Result<String, UmeError>;
    async fn send_push(&self,  msg: PushMessage)  -> Result<String, UmeError>;
    async fn delivery_status(&self, message_id: &str)
        -> Result<DeliveryStatus, UmeError>;
}
```

```
// Available implementations:
// - SmtpCommDriver     — direct SMTP (Postfix, SES SMTP relay)
// - AwsSesDriver       — AWS SES via SDK
// - SendgridDriver     — SendGrid HTTP API
// - TwilioDriver       — Twilio SMS + WhatsApp
// - SlackCommDriver    — Slack Incoming Webhooks
// - TeamsCommDriver    — Microsoft Teams Incoming Webhooks

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct EmailMessage {
    pub to:       Vec<EmailAddress>,
    pub cc:       Vec<EmailAddress>,
    pub from:     EmailAddress,
    pub subject:  String,
    pub html_body: String,
    pub text_body: String,
    pub reply_to: Option<EmailAddress>,
    pub headers:  HashMap<String, String>,
    pub attachments: Vec<Attachment>,
}
```

10.5 Event Broker Driver

Rust

```
#[async_trait]
pub trait BusBackend: Send + Sync {
    async fn publish(&self, envelope: EventEnvelope)  -> Result<(), UmeError>;
    async fn subscribe(&self, pattern: &str,
                       handler: Arc<dyn EventHandler>)
                       -> Result<SubscriptionHandle, UmeError>;
    async fn is_duplicate(&self, key: &str)           -> Result<bool, UmeError>;
    async fn flush_and_close(&mut self)               -> Result<(), UmeError>;
}

// In-memory backend: topic → bounded channel (DashMap<String, Sender>)
pub struct InMemoryBus {
    topics:   Arc<DashMap<String, Vec<Arc<dyn EventHandler>>>>,
    dedup:    Arc<DashMap<String, DateTime<Utc>>>,
    config:   BusConfig,
}

// NATS backend: thin wrapper around async-nats crate
pub struct NatsBus {
    client: async_nats::Client,
    config: NatsConfig,
}
```

10.6 Driver Health & Circuit Breaker

The DeviceBus monitors all drivers and applies circuit breaker logic. A broken circuit prevents modules from issuing calls that will fail immediately — reducing load during backend degradation.

Rust

```
// ume_kernel/src/device_bus.rs — circuit breaker per driver
```

```
#[derive(Debug, Clone, PartialEq, Eq)]
enum CircuitState { Closed, Open { until: DateTime<Utc> }, HalfOpen }

struct DriverCircuit {
    state:          CircuitState,
    failure_count:  u32,
    success_count:  u32,   // in HalfOpen state
    config:         CircuitBreakerConfig,
}

impl DeviceBus {
    pub async fn call<D, F, T>(&self, device_id: &str, f: F)
        -> Result<T, UmeError>
    where
        F: FnOnce(Arc<D>) -> BoxFuture<'_, Result<T, UmeError>>,
        D: DeviceDriver + 0x27sized,
    {
        let circuit = self.circuits.entry(device_id.to_string())
            .or_default();

        // Reject immediately if circuit is open and cooldown not elapsed
        if let CircuitState::Open { until } = circuit.state {
            if Utc::now() < until {
                return Err(UmeError::DriverUnavailable {
                    driver_id: device_id.to_string() });
            }
            circuit.state = CircuitState::HalfOpen;
        }

        let driver = self.get::<D>(device_id)?;
        match f(driver).await {
            Ok(result) => {
                circuit.record_success();
                Ok(result)
            }
            Err(e) => {
                circuit.record_failure();
                Err(e)
            }
        }
    }
}
```

11. Deployment & Java Client Design

11.1 Deployment Topology

Mode	Components	Config File	Suitable For
Embedded	Single ume_runtime binary + SQLite	ume.embedded.toml	Solo operators; development; demos
Compose	ume_server + PostgreSQL 16 + Redis 7 containers	docker-compose.yml	Teams up to ~200 users; self-hosted
Kubernetes	ume_server Deployment (2+ replicas) + managed PG + Redis Cluster	helm/values.yaml	Enterprise; >500 users; HA required

11.2 Docker Image Design

Rust

```
# ume_server Dockerfile (multi-stage)

# Stage 1: builder
FROM rust:1.75-slim AS builder
WORKDIR /build
COPY Cargo.toml Cargo.lock ./
COPY ume_core      ./ume_core
COPY ume_kernel    ./ume_kernel
COPY ume_modules   ./ume_modules
COPY ume_server    ./ume_server
RUN cargo build --release -p ume_server

# Stage 2: runtime (distroless for minimal attack surface)
FROM gcr.io/distroless/cc-debian12
COPY --from=builder /build/target/release/ume_server /usr/local/bin/ume_server

# Non-root user
USER 1000:1000

EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=5s CMD ["/usr/local/bin/ume_server", "health"]
ENTRYPOINT ["/usr/local/bin/ume_server"]

# Image characteristics:
# Base size:  ~30 MB (distroless cc)
# Binary:     ~18 MB (stripped release build)
# Total:      ~48 MB
# No shell, no package manager, no OS vulnerabilities
```

11.3 Kubernetes Helm Chart Structure

Rust

```
helm/ume/
├── Chart.yaml
├── values.yaml                  # user-configurable values
├── values.production.yaml       # production overrides example
```

```
└── templates/
    ├── deployment.yaml          # ume_server Deployment
    ├── service.yaml             # ClusterIP Service
    ├── ingress.yaml             # Ingress (nginx or AWS ALB)
    ├── hpa.yaml                 # HorizontalPodAutoscaler
    ├── pdb.yaml                 # PodDisruptionBudget
    ├── configmap.yaml           # Non-secret config
    ├── secret.yaml              # Secret references (ext-secrets)
    ├── serviceaccount.yaml
    ├── rbac.yaml                # Kubernetes RBAC for service account
    ├── migration-job.yaml       # pre-install Job: run migrations
    └── NOTES.txt

# Key HPA configuration (values.yaml):
# autoscaling:
#   enabled: true
#   minReplicas: 2
#   maxReplicas: 20
#   targetCPUUtilizationPercentage: 70
#   targetEventQueueDepth: 5000   # custom metric via Prometheus adapter
```

11.4 Environment Variable Reference

Variable	Required	Default	Description
UME_ORG_ID	Yes	—	Organization UUID; must match DB on restart
UME_LOG_LEVEL	No	INFO	Log level: TRACE|DEBUG|INFO|WARN|ERROR
UME_SERVER_PORT	No	8080	HTTP listen port
UME_TLS_CERT_PATH	Prod only	—	TLS certificate PEM file path
UME_TLS_KEY_PATH	Prod only	—	TLS private key PEM file path
UME_DB_DSN	Yes (PG)	—	PostgreSQL DSN: postgres://user:pass@host/db
UME_DB_POOL_SIZE	No	10	PostgreSQL connection pool size
UME_REDIS_URL	No	—	Redis URL; enables hot cache tier
UME_EVENTS_BACKEND	No	inmemory	inmemory|nats|kafka|rabbitmq
UME_EVENTS_URL	Cond.	—	Broker connection URL (required if not inmemory)
UME_JWT_KEY_ID	No	ume-default	Vault key ID for JWT signing
UME_RATE_LIMIT_RPM	No	1000	API rate limit: requests per minute per client
UME_AUDIT_RETENTION_YEARS	No	7	Audit record retention in years
UME_MODULES_DIR	No	—	Path to custom module .so directory

11.5 Java Client Architecture

The Java client suite (GUI, Shell, Watch) is a single JAR built with Bazel. It communicates exclusively via the ume_server HTTP API and WebSocket. Three launch modes share a common HTTP client and domain model layer.

Rust

```
clients/java/
├── BUILD.bazel
├── src/main/java/io/ume/client/
│   ├── Main.java                    # entry point; selects mode
│   ├── mode/
│   │   ├── GuiMode.java             # JavaFX full GUI application
│   │   ├── ShellMode.java           # interactive REPL (JLine3)
│   │   └── WatchMode.java           # live dashboard (terminal TUI)
│   ├── api/
│   │   ├── UmeApiClient.java        # HTTP client (OkHttp3)
│   │   ├── UmeWebSocketClient.java  # WebSocket client (OkHttp3 WS)
│   │   ├── AuthManager.java         # token lifecycle: login, refresh, store
│   │   └── endpoints/               # per-module endpoint classes
│   │       ├── FinanceEndpoints.java
│   │       ├── HrEndpoints.java
│   │       └── ...                  # one per module
│   ├── model/                       # Java domain model (mirrors Rust types)
│   │   ├── finance/JournalEntry.java
│   │   ├── hr/Employee.java
│   │   └── ...                      # one package per module domain
│   ├── gui/
│   │   ├── UmeApplication.java      # JavaFX Application subclass
│   │   ├── screens/                 # one Screen per module
│   │   │   ├── FinanceScreen.java
│   │   │   ├── HrScreen.java
│   │   │   └── ...                  # 40+ screens
│   │   ├── components/              # reusable widgets (table, form, chart)
│   │   └── theme/UmeTheme.java      # CSS + color constants
│   └── watch/
│       ├── Dashboard.java           # terminal dashboard layout
│       └── widgets/                 # Panels: KPIs, alerts, module health
```

Rust

```
// AuthManager.java — token lifecycle
public class AuthManager {
    private String accessToken;
    private String refreshToken;
    private Instant accessExpiry;

    /** Return a valid access token, refreshing silently if needed. */
    public synchronized String bearerToken() throws UmeApiException {
        if (accessToken == null) throw new UmeApiException("Not authenticated");
        if (Instant.now().isAfter(accessExpiry.minusSeconds(30))) {
            refresh();  // proactive refresh 30s before expiry
        }
        return "Bearer " + accessToken;
    }

    private void refresh() throws UmeApiException {
        var response = apiClient.post("/v1/auth/refresh",
            Map.of("refresh_token", refreshToken));
        this.accessToken = response.getString("access_token");
        this.accessExpiry = parseExpiry(accessToken);
    }
}
```

```
// WatchMode — real-time dashboard via WebSocket
public class WatchMode {
    public void run() throws Exception {
        var ws = new UmeWebSocketClient(config.serverUrl(), authManager);
        ws.connect();

        // Subscribe to key event topics
        ws.subscribe("risk.kri.*");
        ws.subscribe("finance.period.*");
        ws.subscribe("kernel.module.*");
        ws.subscribe("security.incident.*");

        var dashboard = new Dashboard(terminal);
        ws.onEvent(envelope -> {
            dashboard.updatePanel(envelope.topic(), envelope.payload());
            dashboard.render();
        });

        dashboard.awaitExit();
        ws.disconnect();
    }
}
```

11.6 SDK — Custom Module Scaffold

The ume_sdk crate provides everything a developer needs to build, test, and publish a custom module. The scaffold generator produces a complete, compilable module skeleton.

Rust

```
# Generate a new custom module
$ cargo run -p generate_custom_module -- \
    --vendor acme \
    --name fleet_management \
    --domain operations \
    --output ./my_modules/

# Generated structure:
my_modules/acme_fleet_management/
├── Cargo.toml
├── src/
│   ├── lib.rs              # exports FleetManagementModule
│   ├── module.rs           # KernelModule impl (all lifecycle methods)
│   ├── config.rs           # FleetConfig + ConfigDescriptor declarations
│   ├── domain/
│   │   └── vehicle.rs      # Vehicle, Trip domain types (from ume_core template)
│   ├── repository/
│   │   └── vehicle_repo.rs # VehicleRepository trait + stub impl
│   ├── services/
│   │   └── fleet_service.rs # Business logic service stub
│   └── permissions.rs      # FLEET_PERMISSIONS constant array stub
├── tests/
│   └── integration_tests.rs # Uses MockKernel — tests pass immediately
└── migrations/
    └── fleet_0001_initial.sql  # CREATE TABLE fleet.vehicles stub
```

Rust

```rust
// ume_sdk/src/mock.rs — MockKernel for unit/integration testing

pub struct MockKernel {
    pub audit_log:    Vec<AuditRecord>,
    pub events:       Vec<EventEnvelope>,
    pub permissions:  HashMap<String, bool>,  // preset per test
    pub storage:      Arc<InMemoryStorageDriver>,
}

#[async_trait]
impl KernelFacade for MockKernel {
    async fn assert_can(&self, subject: &SubjectId, permission: &str,
                        _resource: Option<&str>) -> Result<(), UmeError> {
        if *self.permissions.get(permission).unwrap_or(&true) {
            Ok(())
        } else {
            Err(UmeError::AuthorizationFailed {
                subject: subject.to_string(),
                permission: permission.to_string(),
            })
        }
    }

    async fn publish(&self, topic: &str, payload: serde_json::Value,
                     _meta: EventMetadata) -> Result<EventId, UmeError> {
        self.events.push(EventEnvelope { topic: topic.into(), payload,
            ..Default::default() });
        Ok(EventId::new())
    }

    // ... full trait implementation with in-memory storage
}
```

Appendix A: Complete Event Catalogue

The following table lists all domain events emitted by the UME platform. Subscribers use glob patterns to subscribe to event families.

Topic	Publisher Module	Payload Fields	Common Subscribers
admin.org_unit.created	MOD-01	org_unit_id, name, parent_id, manager_id	Analytics, HR, Finance
admin.policy.published	MOD-01	policy_id, title, affected_unit_ids	Comms, GRC, Learning
finance.journal.posted	MOD-14	entry_id, period, entity_id, total_debit	Analytics, GRC
finance.invoice.created	MOD-14	invoice_id, vendor_id, amount, currency	Supply Chain, Comms
finance.invoice.approved	MOD-14	invoice_id, approver_id, approved_at	Finance (payment queue)
finance.period.locked	MOD-14	period, entity_id, locked_by	Analytics, GRC, Risk
finance.budget.variance_alert	MOD-14	budget_id, account_code, variance_pct	Risk, Analytics, Comms
hr.employee.hired	MOD-16	employee_id, org_unit_id, start_date, manager_id	Process (onboarding), Security, Comms
hr.employee.terminated	MOD-16	employee_id, end_date, reason	Process (offboarding), Security, Finance
hr.leave.approved	MOD-16	leave_id, employee_id, start_date, end_date	Schedule, Work, Comms
hr.payroll.completed	MOD-16	payroll_run_id, period, employee_count	Finance (journal), Analytics
chombo.entity.evaluated	MOD-13	entity_id, alert_count, critical_alerts	Board, Risk, Analytics
chombo.filing.due	MOD-13	entity_id, filing_type, due_date, lead_days	Comms, Process, Board
chombo.filing.overdue	MOD-13	entity_id, filing_type, due_date	Comms, Risk, Board
grc.control.failed	MOD-15	control_id, obligation_id, finding	Risk, Board, Comms
grc.audit.report_issued	MOD-15	audit_id, finding_count, report_url	Board, Risk, Comms
risk.kri.amber_threshold	MOD-33	kri_id, risk_id, value, threshold	Comms, Analytics, Board
risk.kri.red_threshold	MOD-33	kri_id, risk_id, value, threshold	Comms, Analytics, Board, Risk
risk.risk.appetite_breached	MOD-33	risk_id, residual_score, appetite_limit	Board, Comms
security.identity.login	MOD-36	identity_id, ip_address, mfa_used	Audit (already written)
security.identity.lockout	MOD-36	identity_id, reason, unlock_at	Comms, Audit

security.incident.created	MOD-36	incident_id, classification, priority	IT, Comms
security.incident.p1_declared	MOD-36	incident_id, description	Board, Comms, Risk
sc.inventory.reorder_triggered	MOD-37	item_id, location_id, current_qty, order_qty	Supply Chain (PO create), Comms
sc.po.received	MOD-37	po_id, lines_received	Finance (3-way match)
sales.opportunity.won	MOD-34	opportunity_id, value, customer_id	Finance (invoice), Analytics, CRM
sales.opportunity.lost	MOD-34	opportunity_id, reason, competitor	Analytics, CRM
work.item.completed	MOD-40	item_id, completed_by, project_id	Analytics, Portfolio
process.workflow.completed	MOD-29	instance_id, definition_id, context	Module-specific consumers
process.task.created	MOD-29	instance_id, step_id, assignee, form_id, due_by	Work (task creation), Comms
soko.campaign.evaluated	MOD-22	campaign_id, insight_count, top_action	Analytics
audit.record.created	Kernel	audit_record_id, actor, action, outcome	SIEM webhook
kernel.module.started	Kernel	module_id, version	Enterprise Mgmt, Analytics
kernel.module.stopped	Kernel	module_id, reason	Enterprise Mgmt, Comms (alert)
kernel.module.degraded	Kernel	module_id, health_message	Enterprise Mgmt, Comms (alert)

Appendix B: Database Index Reference

Critical indexes that must be present for acceptable query performance. All indexes are created in migration scripts.

SQL / DDL

```
-- Universal: every org_id column in every table
CREATE INDEX idx_{table}_org_id ON {schema}.{table}(org_id);

-- Finance: frequent query patterns
CREATE INDEX idx_journal_entries_period ON finance.journal_entries(org_id,
entity_id, period);
CREATE INDEX idx_journal_entries_source ON finance.journal_entries(source_module,
source_id);
CREATE INDEX idx_journal_lines_account  ON finance.journal_lines(org_id,
account_code);
CREATE INDEX idx_invoices_vendor        ON finance.invoices(org_id, vendor_id,
status);
CREATE INDEX idx_invoices_due_date      ON finance.invoices(org_id, due_date) WHERE
status = 'Unpaid';

-- HR: common lookups
CREATE INDEX idx_employees_org_unit ON hr.employees(org_id, org_unit_id, status);
CREATE INDEX idx_employees_manager  ON hr.employees(manager_id);
CREATE INDEX idx_employees_status   ON hr.employees(org_id, status);
```

```
-- Risk: KRI monitoring (called on scheduler tick)
CREATE INDEX idx_kris_risk_id ON risk.kris(risk_id, org_id);

-- Audit: compliance query patterns (time range by actor)
CREATE INDEX idx_audit_actor_at   ON audit.audit_records(org_id, actor, at DESC);
CREATE INDEX idx_audit_action     ON audit.audit_records(org_id, action, at DESC);
CREATE INDEX idx_audit_resource   ON audit.audit_records(org_id, resource_type,
resource_id);

-- Work: board and backlog queries
CREATE INDEX idx_work_items_assignee ON work.items(org_id, assignee_id, status);
CREATE INDEX idx_work_items_sprint   ON work.items(sprint_id, status);
CREATE INDEX idx_work_items_project  ON work.items(project_id, status);
```

Appendix C: Dependency Graph (Module Dependencies)

Module	Depends On Modules	Depends On Kernel Services
MOD-01 Administration	None	Storage, RBAC, Audit, Events
MOD-02 Analytics	All modules (event subscriber)	Storage, Events, Executor(analytics)
MOD-03 Backup	MOD-36 Security (encryption)	Storage, Executor(background)
MOD-04 Board	MOD-14 Finance, MOD-33 Risk, MOD-15 GRC	Storage, RBAC, Audit, Events
MOD-13 Chombo	None	Storage, RBAC, Audit, Events, Executor(ops)
MOD-14 Finance	MOD-01 Admin, MOD-13 Chombo	Storage, RBAC, Audit, Events, Executor(finance)
MOD-15 GRC	MOD-14 Finance, MOD-18 IT, MOD-36 Security	Storage, RBAC, Audit, Events
MOD-16 HR	MOD-01 Admin, MOD-20 Learning	Storage, RBAC, Audit, Events, Executor(hr)
MOD-22 Soko	MOD-08 CRM, MOD-34 Sales	Storage, RBAC, Audit, Events, Executor(ops)
MOD-29 Process	MOD-40 Work, MOD-07 Comms	Storage, RBAC, Events, Executor(ops)
MOD-33 Risk	MOD-15 GRC	Storage, RBAC, Audit, Events, Executor(background)
MOD-34 Sales	MOD-08 CRM, MOD-14 Finance	Storage, RBAC, Audit, Events
MOD-36 Security	None	Storage, RBAC, Audit, Events, CryptoVault
MOD-37 Supply Chain	MOD-14 Finance, MOD-01 Admin	Storage, RBAC, Audit, Events
MOD-40 Work	MOD-01 Admin, MOD-27 Portfolio	Storage, RBAC, Audit, Events

Appendix D: Glossary

Term	Definition
Composition Root	The single location (UmeKernel::boot) where all dependencies are wired together.

DeviceBus	Kernel subsystem that registers, manages, and mediates access to all device drivers.
Domain Event	An immutable fact published to the event bus after a state-changing operation completes.
DLQ (Dead-Letter Queue)	Destination for events that could not be delivered after all retries are exhausted.
EncryptedBlob	Struct containing ciphertext, nonce, auth tag, and key reference — produced by CryptoVault.
EventEnvelope	Wrapper around a domain event payload that adds routing, audit, and tracing metadata.
Filter	Type-safe query predicate used by modules to query StorageManager without writing SQL.
KernelFacade	The narrow interface given to each module granting controlled access to kernel services.
KernelModule	The Rust trait all organization modules must implement to participate in the kernel.
ModuleDescriptor	Self-description of a module: ID, dependencies, permissions, event subscriptions.
ModuleState	Lifecycle state enum: Registered → Initializing → Starting → Running → Degraded → Error → Stopping → Stopped.
Newtype Wrapper	Rust pattern: struct OrgUnitId(Uuid) — prevents accidentally passing wrong ID type.
Optimistic Lock	Concurrency control via version counter: rejected if version in DB != version in request.
Repository	Domain-typed wrapper around StorageManager that exposes module-specific query methods.
Soft Delete	Marking deleted_at on a record instead of issuing DELETE — preserves audit trail and referential integrity.
Strategy Pack	A rule-based evaluator in Soko that returns marketing insights and actions for a campaign.
SubjectId	Identifier for any actor in the RBAC system: User, ServiceAccount, or ApiKey.
SupervisorEngine	Kernel background task that monitors module health and applies restart policies.
UmeError	The exhaustive error enum covering all failure modes; all fallible functions return Result<T,UmeError>.
ume_core	Foundational crate: all shared types, traits, and contracts; depends on nothing in the workspace.